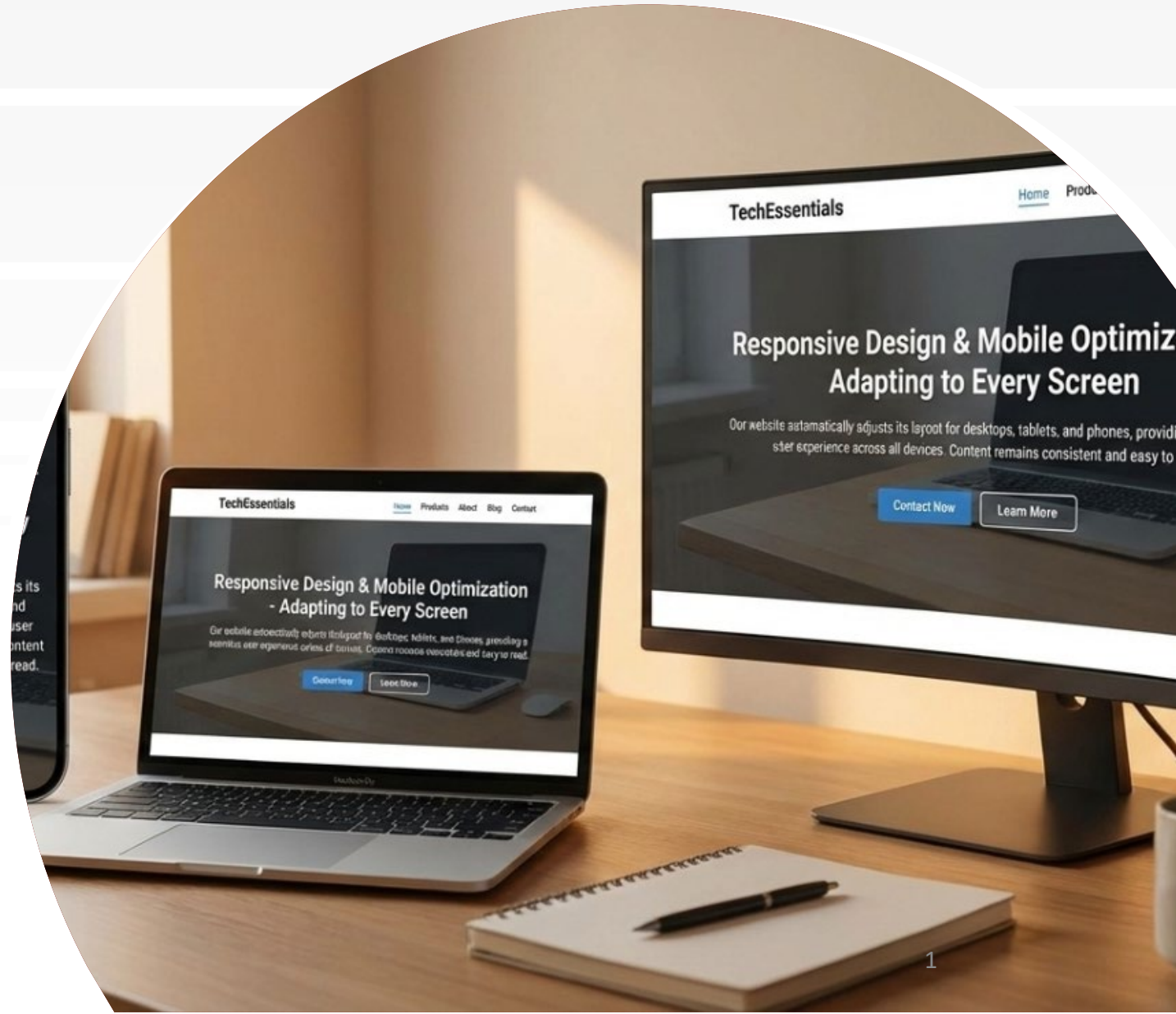


INF1005

Topics for Week 02:

- Cascading Style Sheets (CSS)
- Responsive Design with Bootstrap





```
body {  
  font: x-s  
  background  
  color: bla  
  margin: 0;  
  padding: 0
```

CASCADING STYLE SHEETS

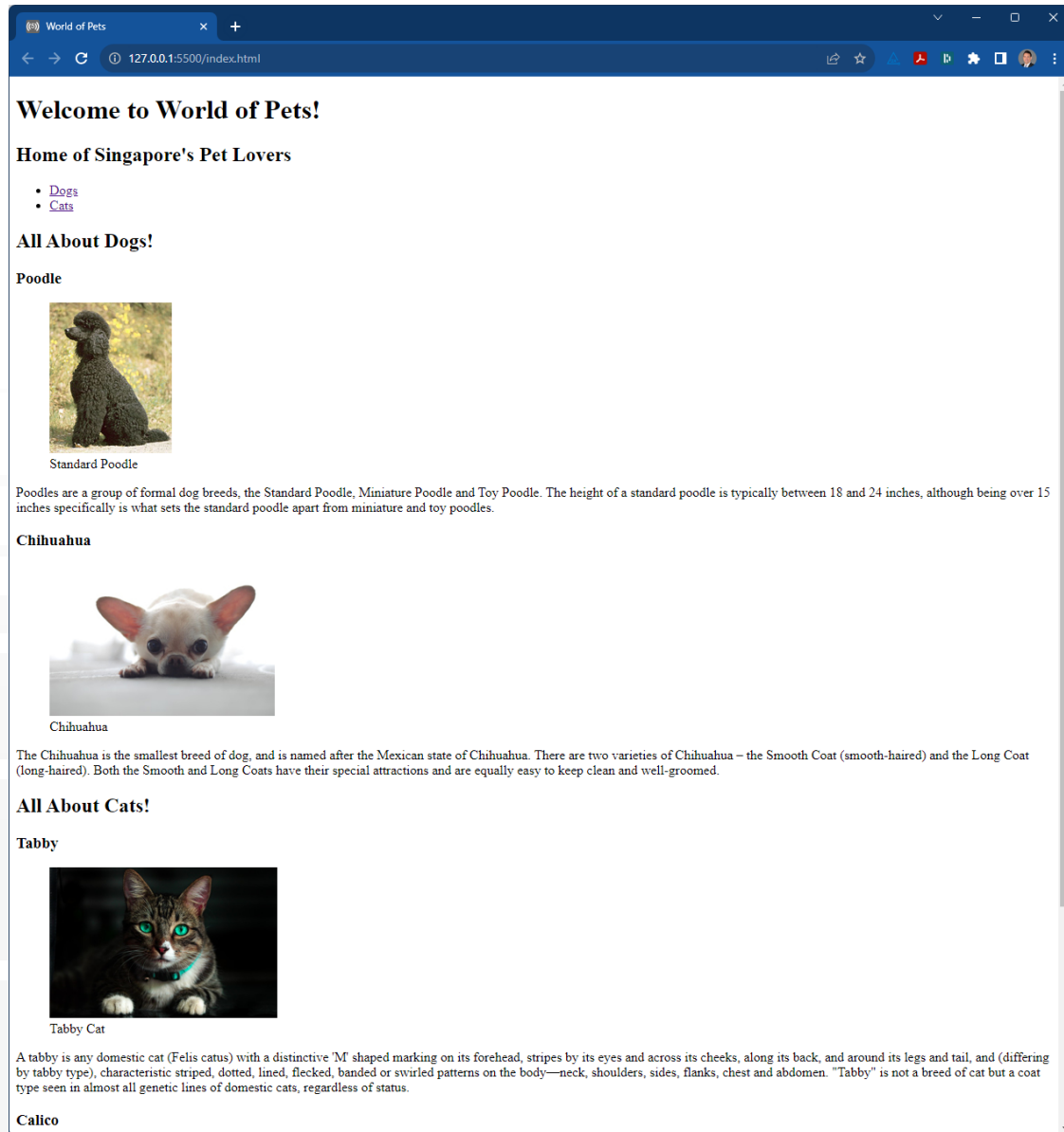
- CSS Rule-sets
 - Selector
 - Property
 - Value
- Including CSS in HTML
 - External
 - Internal (a.k.a. embedded)
 - Inline



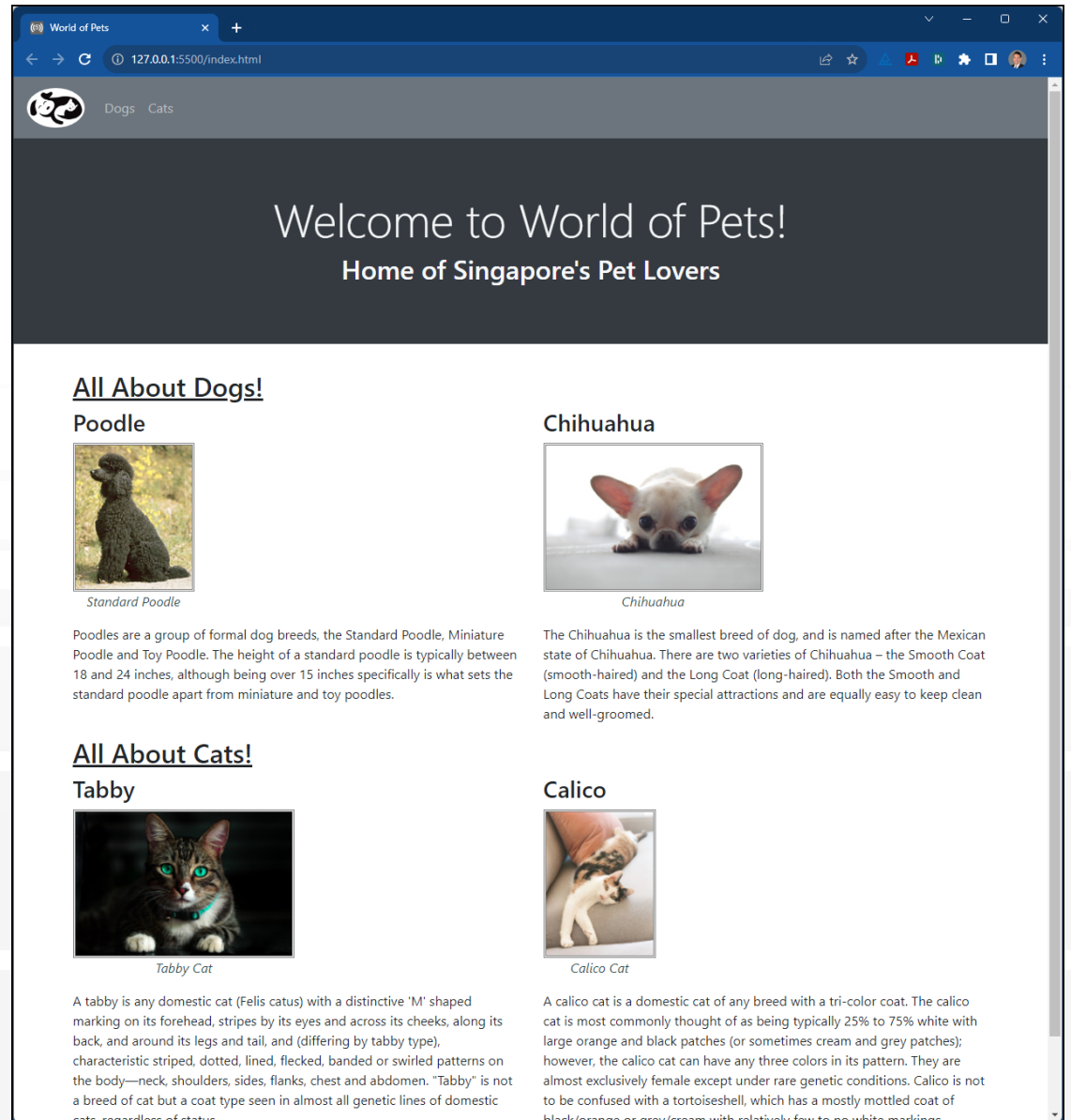
BOOTSTRAP

- Intro to Frameworks and Bootstrap
- Responsive Design
- Including Bootstrap in HTML
- Bootstrap Grid System
- Jumbotron
- Navigation Menu
- Misc. Bootstrap Features

Before



After



► **Additional Materials for Self-study**

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar



Chapter 4

CSS 1: Selectors and Basic Styling

In this chapter you will learn . . .

- The rationale for CSS
- The syntax of CSS
- Where CSS styles can be located
- The different types of CSS selectors
- What the CSS cascade is and how it works
- The CSS box model
- CSS text styling

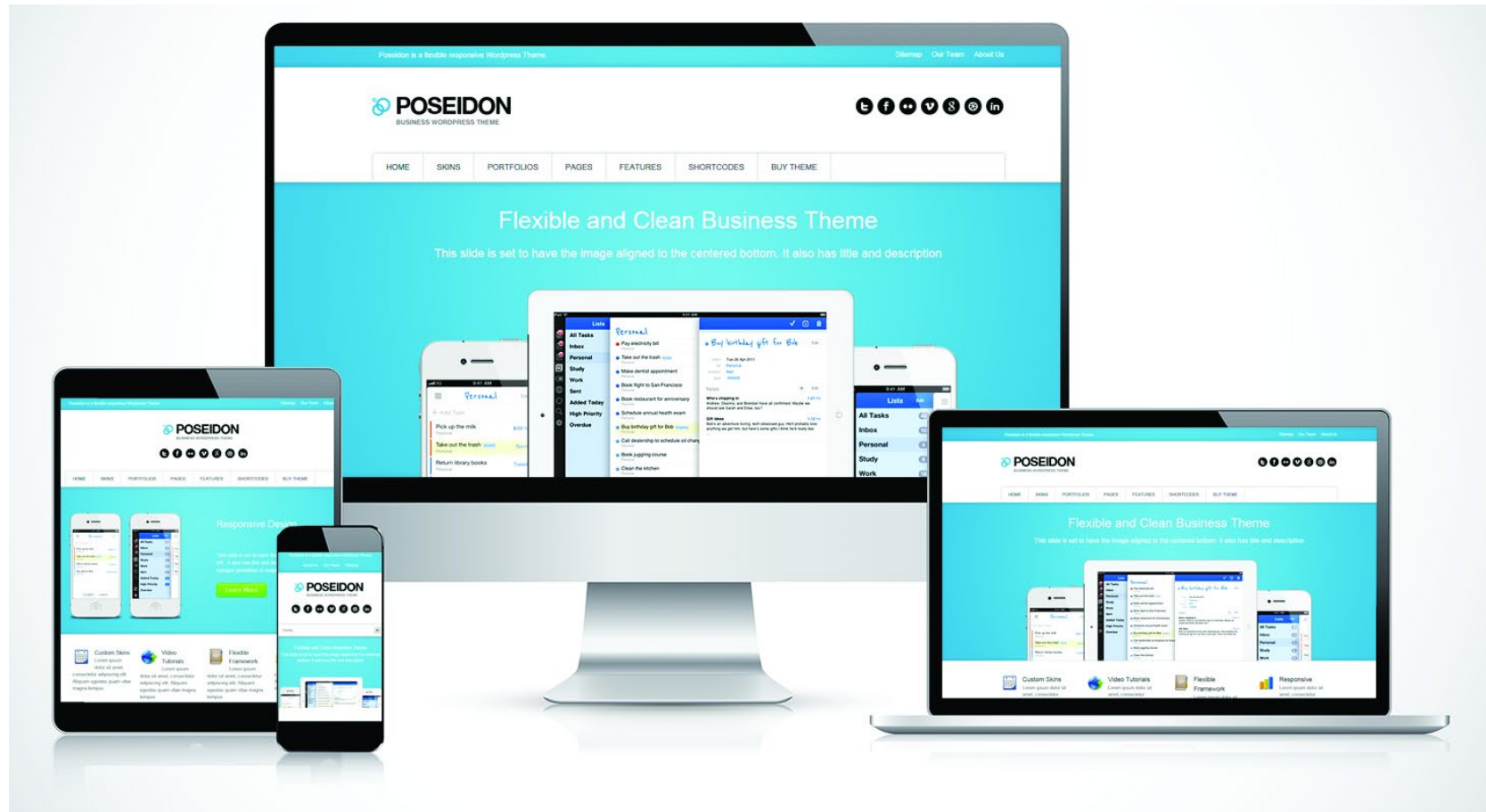
What Is CSS?

- CSS is a W3C standard for describing the appearance of HTML elements.
- With CSS, we can assign font properties, colors, sizes, borders, background images, positioning and even animate elements on the page.
- CSS can be added directly to any HTML element (via the style attribute), within the <head> element, or, most commonly, in a separate text file that contains only CSS.

Benefits of CSS

- **Improved control over formatting.**
- **Improved site maintainability.** All formatting can be centralized into one CSS file
- **Improved accessibility.** By keeping presentation out of the HTML, screen readers and other accessibility tools work better.
- **Improved page-download speed.** Each individual HTML file will contain less style information and markup, and thus be smaller.
- **Improved output flexibility.** CSS can be used to adopt a page for different output media. This approach to CSS page design is often referred to as **responsive design**.

CSS allows Responsive Design



CSS Versions

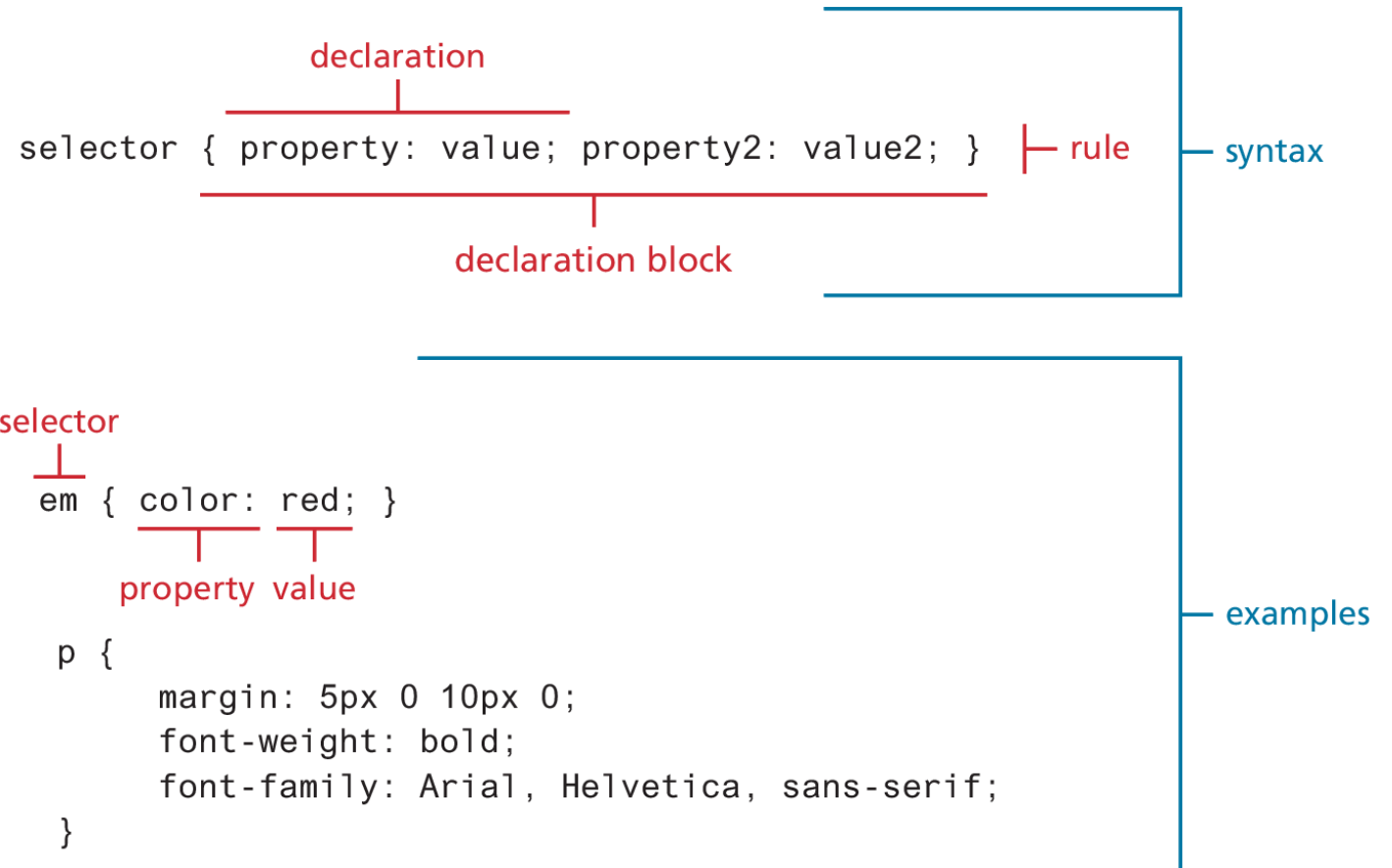
- Style sheets as a way to visually format markup predate the web.
- W3C decided to adopt CSS, and by the end of 1996, the CSS Level 1 Recommendation was published
- A year later, the CSS2 was published
- CSS2.1, did not become an official W3C Recommendation until June 2011
- At the same time the CSS2.1 standard was being worked on, a different group at the W3C was working on a CSS3 draft

Browser Adoption

- Perhaps the most important thing to keep in mind with CSS is that the different browsers have not always kept up with the W3C.
- For this reason, CSS has a reputation for being a somewhat frustrating language.
- In this book, we hope to redress that negative reputation by covering CSS basics and then incrementally introducing ideas until finally we cover modern frameworks that address many of those challenges.

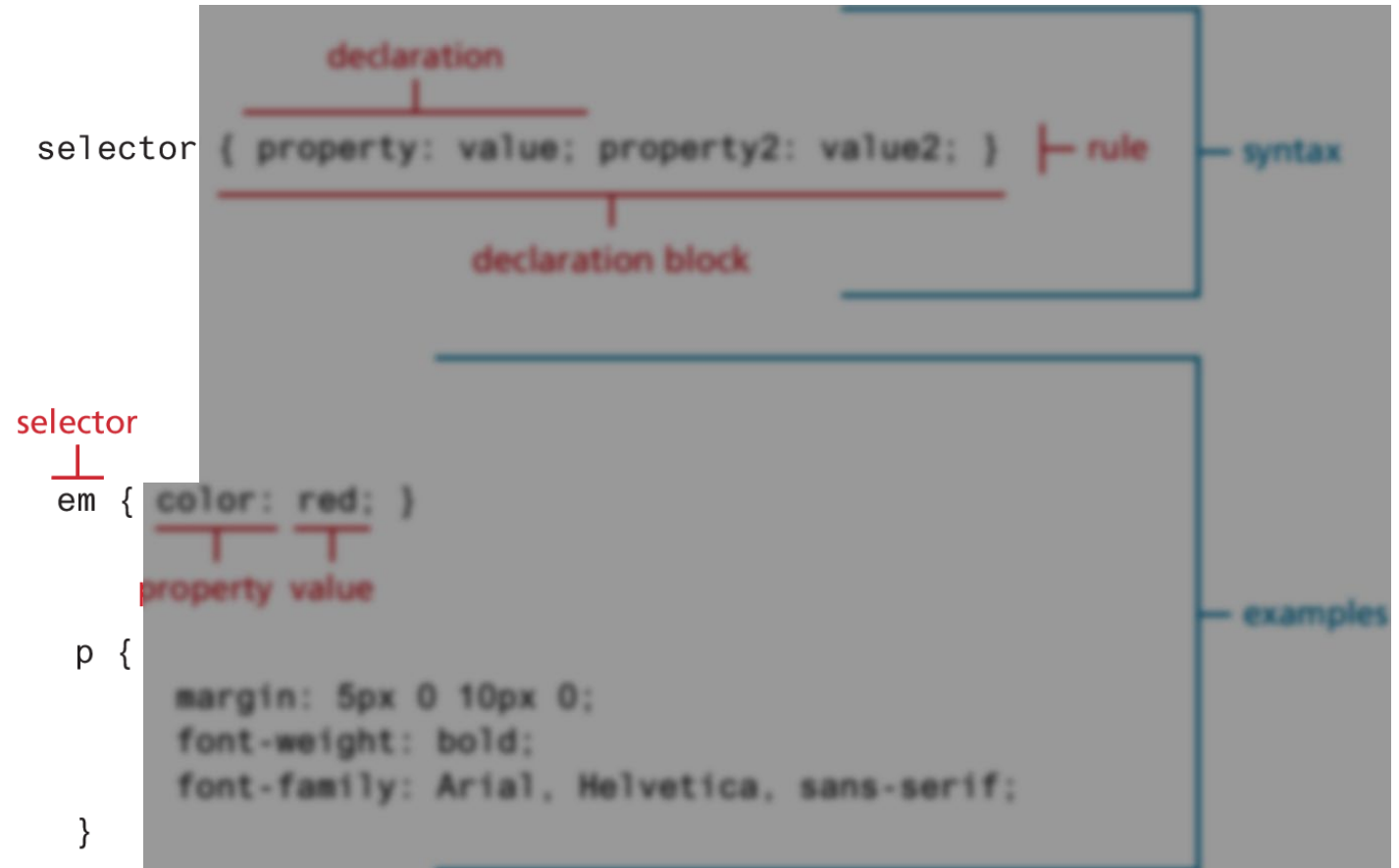
CSS Syntax

- A CSS document consists of one or more **style rules**.
- A rule consists of a **selector** that identifies the HTML element or elements that will be affected, followed by a series of **property:value pairs** called **the declaration**
- The series of declarations is also called the **declaration block**.



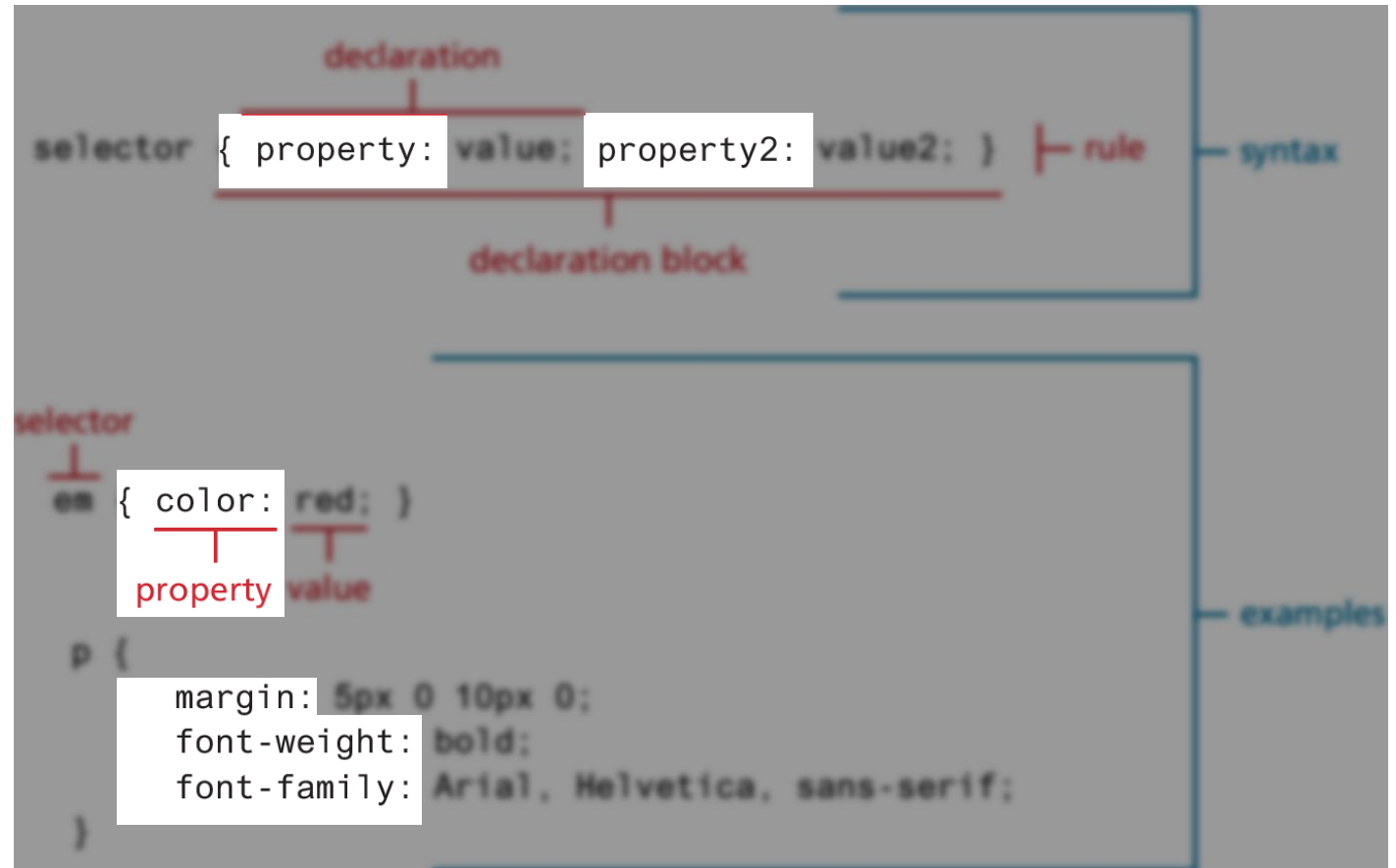
CSS Syntax: Selector

- Every CSS rule begins with a **selector**.
- The **selector** identifies which element or elements in the HTML document will be affected by the declarations in the rule.



CSS Syntax: Properties

- Each individual CSS declaration must contain a **property**
- The CSS2.1 recommendation defines over a hundred different property names!
- Table 4.1 lists many of the most commonly used CSS properties.



CSS Syntax: Values

- Each individual CSS declaration must also contain a **value**
- The unit of any given value is dependent upon the property.
- Some property values are from a predefined list of keywords. Others are values such as length measurements, percentages, numbers without units, color values, and URLs.

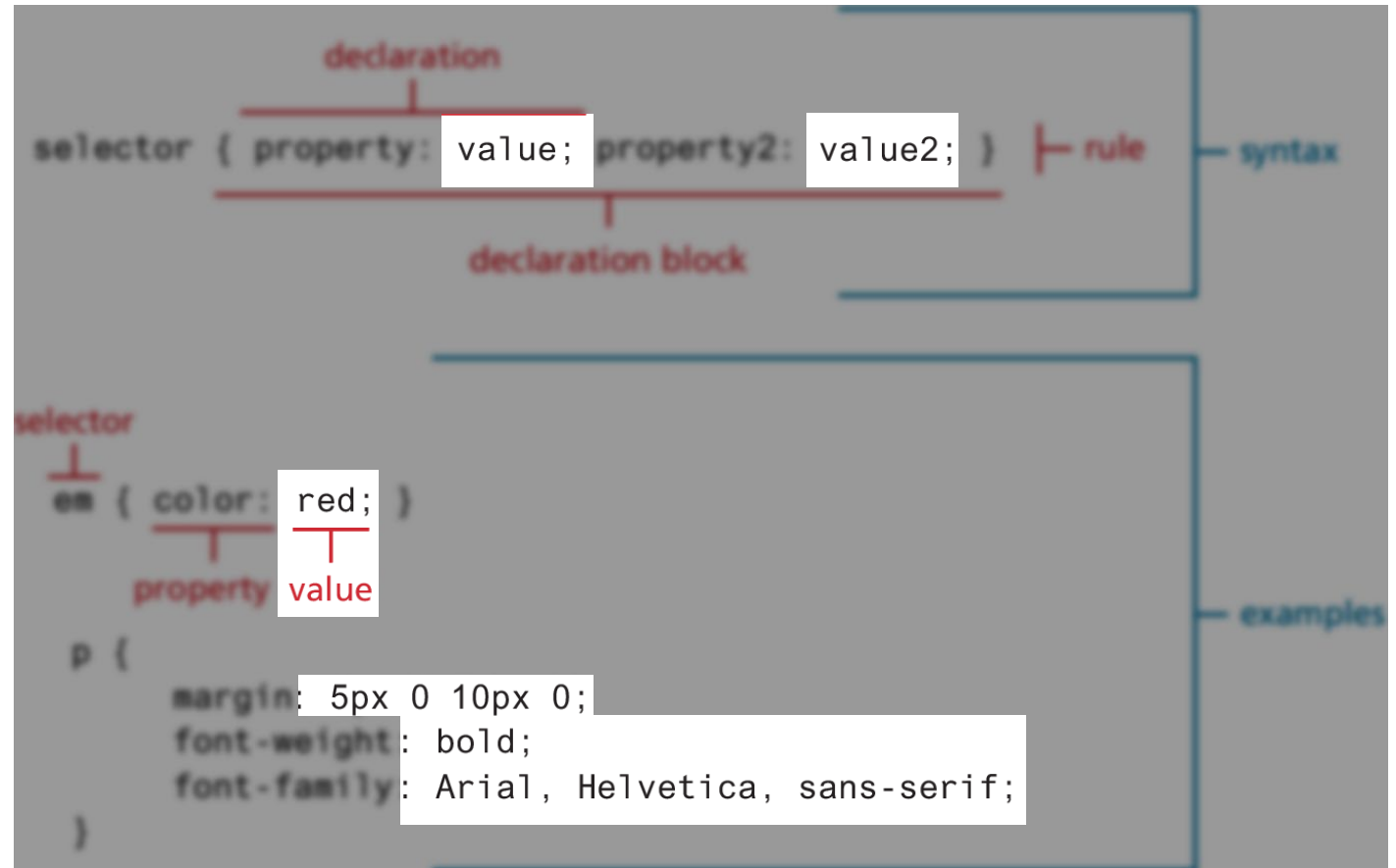


TABLE 4.2 Color Values

Method	Description	Example
Name	Use one of 17 standard color names. CSS3 has 140 standard names.	color: red; color: hotpink; /* CSS3 only */
RGB	Uses three different numbers between 0 and 255 to describe the red, green, and blue values of the color.	color: rgb(255,0,0); color: rgb(255,105,180);
Hexadecimal	Uses a six-digit hexadecimal number to describe the red, green, and blue value of the color	color: #FF0000; color: #FF69B4;
RGBA	This defines a partially transparent background color. The “a” stands for “alpha,” which is a term used to identify a transparency	color: rgba(255,0,0,0.5);
HSL	Allows you to specify a color using Hue Saturation and Light values. This is available only in CSS3. HSLA is also available as well.	color: hsl(0,100%,100%); color: hsla(330,59%,100%,0.5);

Common Units of Measure Values

Units of measure in CSS are either

- **relative units**, in that they are based on the value of something else, or
- **absolute units**, in that they have a real-world size.

Some example measures (See Table 4.3 for more)

- **px** Pixel. In CSS2 this is a relative measure, while in CSS3 it is absolute (1/96 of an inch)
- **em** Equal to the computed value of the font-size property of the element on which it is used.
- **%** A measure that is always relative to another value.
- **in** Inches
- **cm** Centimeters

CSS Comments

NOTE

It is helpful to add comments to your style sheets. Comments take the form:

```
/* comment goes here */
```

It is a common practice to locate related style rules together, and then indicate that they are related via a comment. For instance:

```
/* main navigation */
```

```
nav#main { ... }  
nav#main ul { ... }  
nav#main ul li { ... }
```

```
/* header */
```

```
header { ... }  
h1 { ... }
```

Comments can also be a helpful way to temporarily hide any number of rules, to help with debugging.



Location of Styles

CSS style rules can be located in three different locations.

- Inline Styles
- Embedded Style Sheet
- External Style Sheet

These three are not mutually exclusive, in that you could place your style rules in all three.

Inline Styles

Inline styles are style rules placed within an HTML element via the style attribute

- An inline style only affects the element it is defined within and overrides any other style definitions for properties used
- Notice that a selector is not necessary

```
<h1>Share Your Travels</h1>  
<h2 style="font-size: 24pt">Description</h2>  
...  
<h2 style="font-size: 24pt; font-weight: bold;">Reviews</h2>
```

LISTING 4.1 Inline styles example

Embedded Style Sheet

Embedded style sheets (also called internal styles) are style rules placed within the `<style>` element (inside the `<head>` element of an HTML document)

- While better than inline styles, using embedded styles is also by and large discouraged.

```
<head>
  <meta charset="utf-8">
  <title>Chapter 4</title>
  <style>
    h1 { font-size: 24pt; }
    h2 {
      font-size: 18pt;
      font-weight: bold;
    }
  </style>
</head>
<body>
```

LISTING 4.2 Embedded styles example

External Style Sheet

External style sheets are style rules placed within an external text file with the **.css** extension.

- When you change an external style sheet, all HTML documents that reference that style sheet will automatically use the updated version.
- The browser is able to cache the external style sheet, which can improve performance as well.

```
<head>  
  <meta charset="utf-8">  
  <title>Chapter 4</title>  
  <link rel="stylesheet" href="styles.css" />  
</head>
```

LISTING 4.3 Referencing an external style sheet

Selectors

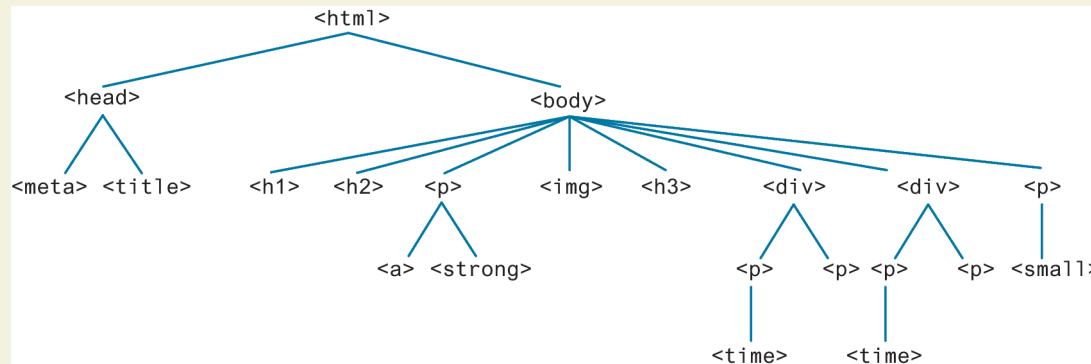
When defining CSS rules, you will need to use a selector.

- Selectors tell the browser which elements will be affected by the property values
- Selectors allow you to select individual or multiple HTML elements.
- Three basic selector types have been around since the earliest CSS2 specification.
 - Element Selectors
 - Class Selectors
 - Id Selectors

The Document Object Model (DOM)

NOTE

The **Document Object Model** (DOM) is how a browser represents an HTML page internally. This DOM is akin to a tree representing the overall hierarchical structure of the document.



In the last chapter, we learned some of the familial terminologies (such as descendants, ancestors, siblings, etc.) in an HTML document. You will at times have to think about the elements in your HTML document in terms of their position within the hierarchy.



Element Selectors

Element selectors select all instances of a given HTML element.

You can also select all elements by using the **universal element selector** (*)

You can select a group of elements by separating the different element names with commas.

```
/* commas allow you to group selectors */
p, div, aside {
    margin: 0;
    padding: 0;
}
/* the above single grouped selector is equivalent to the
following: */
p {
    margin: 0;
    padding: 0;
}
div {
    margin: 0;
    padding: 0;
}
aside {
    margin: 0;
    padding: 0;
}
```

LISTING 4.4 Sample grouped selector

Class and ID Selectors

A **class selector** allows you to simultaneously target different HTML elements regardless of their position in the document tree.

HTML elements labeled with the same class attribute value, can be targeted for styling by using a class selector, which takes the form:

period (.) followed by the class name.

An **ID selector** allows you to target a specific element by its id attribute regardless of its type or position in the document tree.

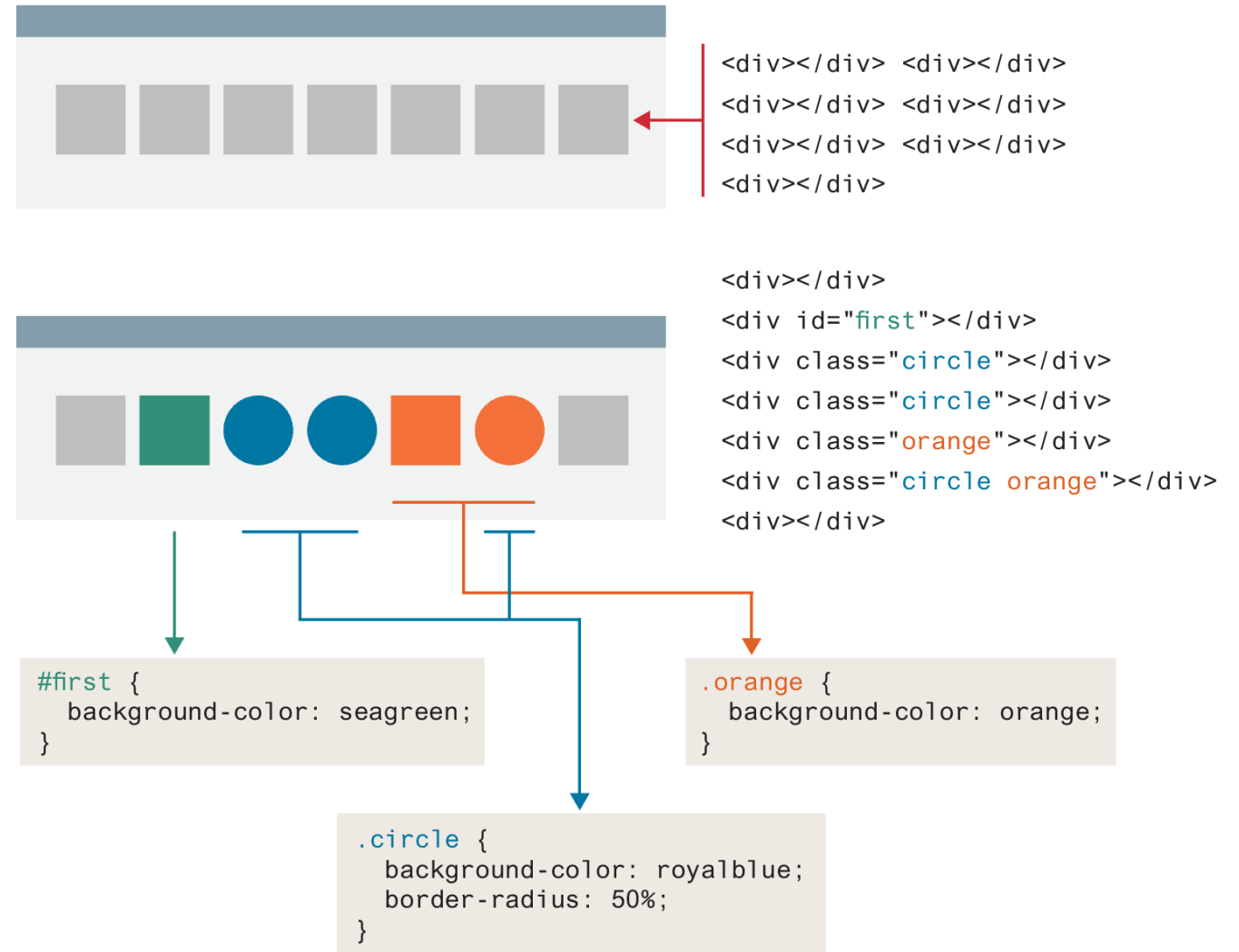
HTML elements labeled with an id attribute, can be targeted for styling by using an id selector, which takes the form:

pound/hash (#) followed by the id name.

Class and ID Selector Example

Consider Figure 4.4. The original HTML can be modified to add class and id values, which are then styled in CSS.

- Id selector **#first** matches the div with id “first”
- Class selectors **.orange** and **.circle**, match all divs with those class values.
- Notice that an element can be tagged with multiple classes.



Attribute Selectors

An **attribute selector** provides a way to select HTML elements either by the presence of an element attribute or by the value of an attribute.

- Attribute selectors can be helpful in the styling of hyperlinks and form elements (which come up in the next chapter)
- These selectors can be combined with the element id and class selectors.
- You use square brackets to create attribute selectors (see Table 4.3)

TABLE 4.4 Attribute Selectors

Matches		Example
[]	A specific attribute.	[title] Matches any element with a title attribute
[=]	A specific attribute with a specific value.	a[title="posts from this country"] Matches any <a> element whose title attribute is exactly "posts from this country"
[~=]	A specific attribute whose value matches at least one of the words in a space-delimited list of words.	[title~="Countries"] Matches any title attribute that contains the word "Countries"
[^=]	A specific attribute whose value begins with a specified value.	a[href^="mailto"] Matches any <a> element whose href attribute begins with "mailto"
[*=]	A specific attribute whose value contains a substring.	img[src*="flag"] Matches any element whose src attribute contains somewhere within it the text "flag"
[\$=]	A specific attribute whose value ends with a specified value.	a[href\$=".pdf"] Matches any <a> element whose href attribute ends with the text ".pdf"

Attribute Selector Example

Here we show an attribute selector style links to show PDF files differently than other links (from Figure 4.5)

```
a[href$=".pdf"] {  
    background: url(pdf_icon.svg) no-repeat left center;  
    padding-left: 19px;  
}
```

This attribute selector selects only those `<a>` elements whose href attribute value ends with the characters `".pdf"`.

```
<a href="abc.html">First link</a>  
<a href="sample.pdf">One PDF</a>  
<a href="another.pdf">Two PDF</a>
```

[First link](#)  [One PDF](#)  [Two PDF](#)

Pseudo-Element and Pseudo-Class Selectors

A **pseudo-element selector** is a way to select something that does not exist explicitly as an element in the HTML document tree.

- You can select the **first line** or **first letter** of any HTML element using a pseudo-element selector.

A **pseudo-class selector** targets either a particular state or a variety of family relationships

- This type of selector is for targeting link states and for adding hover styling for other elements

Common Pseudo-Class and Pseudo-Element Selectors

- **a:link** Selects links that have not been visited.
- **a:visited** Selects links that have been visited.
- **:hover** Selects elements that the mouse pointer is currently above.
- **:active** Selects an element that is being activated by the user. A typical example is a link that is being clicked.
- **:first-child** Selects an element that is the first child of its parent.
- **:first-letter** Selects the first letter of an element.
- **:first-line** Selects the first line of an element.
- Table 4.5 for more common selectors.

Styling a link using pseudo-class selectors

Home Mens Womens Kids **House** Garden Contact `li:hover { ... }`

Home Mens Womens Kids House Garden Contact `li:first-child { ... }`

Home Mens Womens Kids House Garden Contact `li:nth-child(2n) { ... }`

Home Mens Womens Kids House Garden Contact `li:nth-child(2n-1) { ... }`

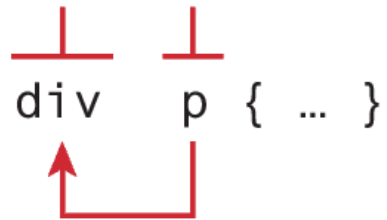
[Arsenal](#)
[Chelsea](#)
[Liverpool](#)
[Manchester United](#)
[West Ham United](#)

`a:link { ... }`
`a:visited { color: royalblue }`
`a:hover { color: lavender; background-color: hotpink }`
`a:active { font-weight: bold }`
`a:link:last-child { text-decoration: none }`

Pseudo selectors can be combined

Syntax of a descendant selection

context selected element


div p { ... }

Selects a <p> element
somewhere
within a <div> element


#main div p:first-child { ... }

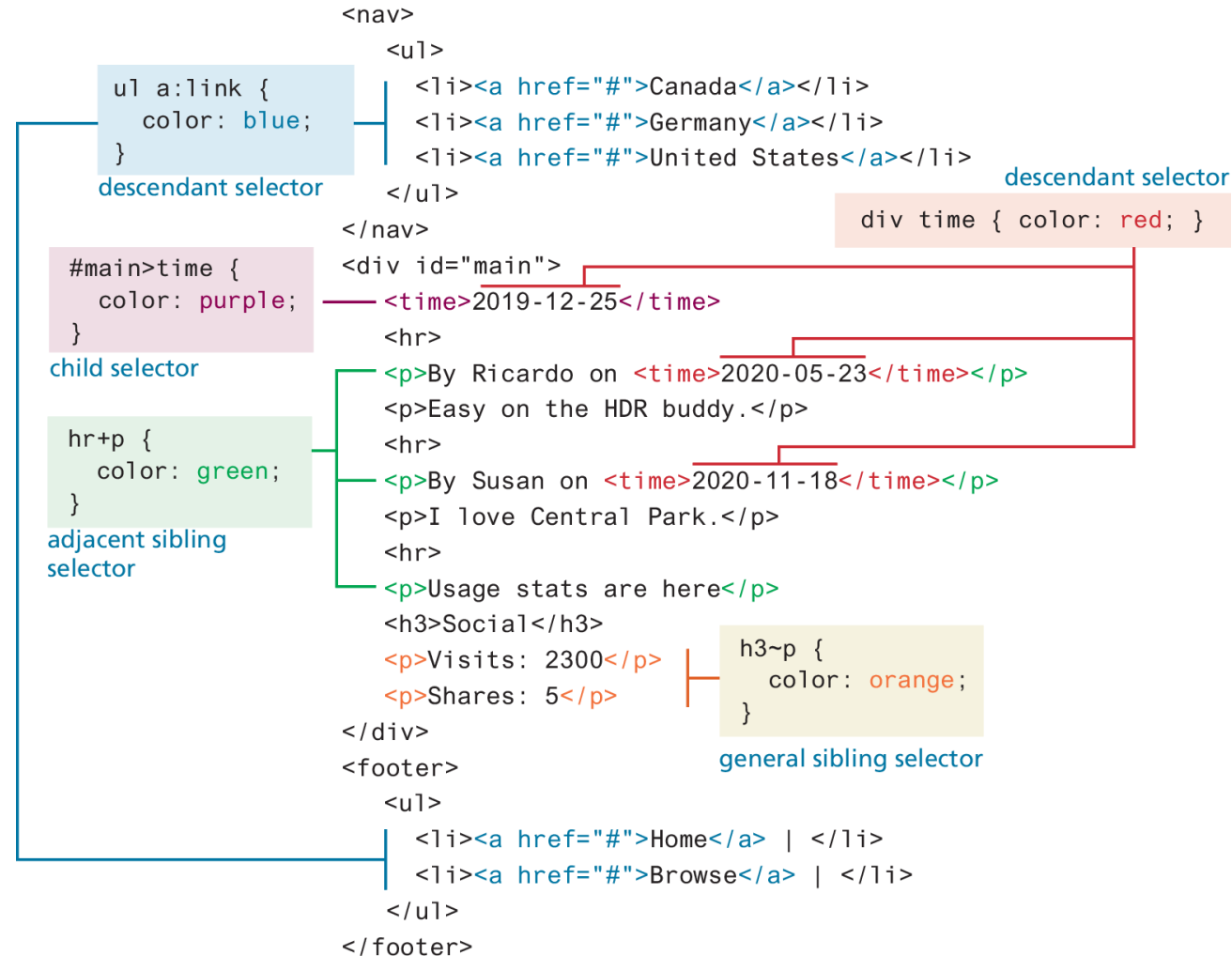
Selects the first <p> element
somewhere within a <div> element
that is somewhere within an element
with an id="main"

Contextual Selectors

A **contextual selector** (in CSS3 also called **combinators**) allows you to select elements based on their *ancestors*, *descendants*, or *siblings*.

- A **descendant selector** matches all elements that are contained within another element. The character used to indicate descendant selection is a space “ ”
- A **child selector** matches a specified element that is a direct child of the specified element. The character used is a “>”
- An **Adjacent selector** matches a specified element that is the next sibling of the specified element. The character used is a “+”
- A **General selector** matches All following elements that shares the same parent as the specified element. The character used is a “~”

Contextual selectors in action



The Cascade: How Styles Interact

The “Cascade” in CSS refers to how conflicting rules are handled.

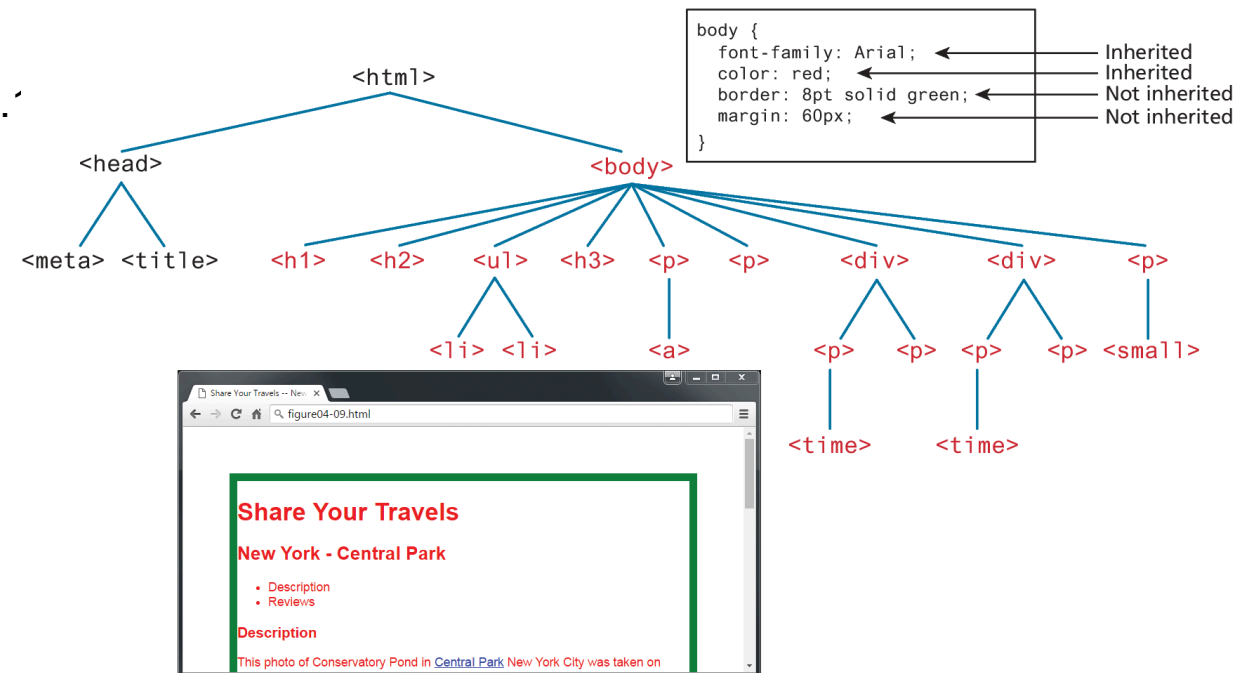
CSS uses the following cascade principles to help it deal with conflicts:

- inheritance,
- specificity, and
- location.

The Cascade: Inheritance

Inheritance is the principle that many CSS properties affect their descendants as well as themselves.

- Font, color, list, and text properties (from Table 4.1) are inheritable;
- Layout, sizing, border, background, and spacing properties are not.
- it is also possible to inherit properties that are normally not inheritable using **inherit**

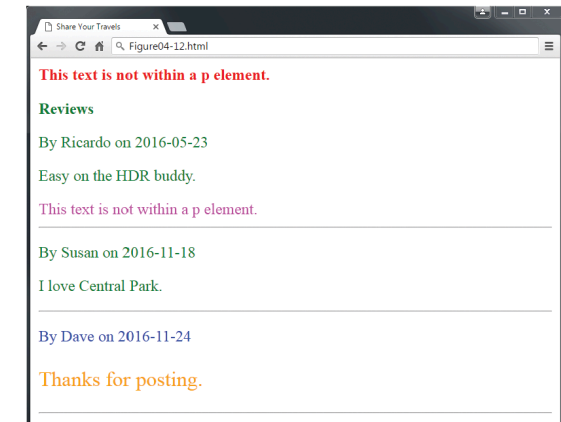
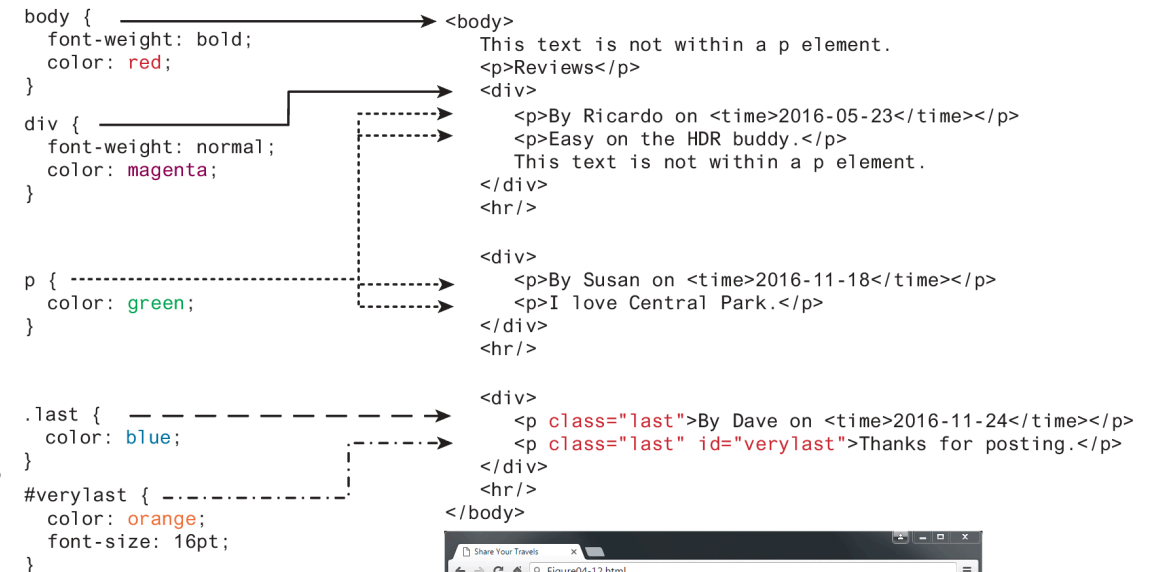


The Cascade: Specificity

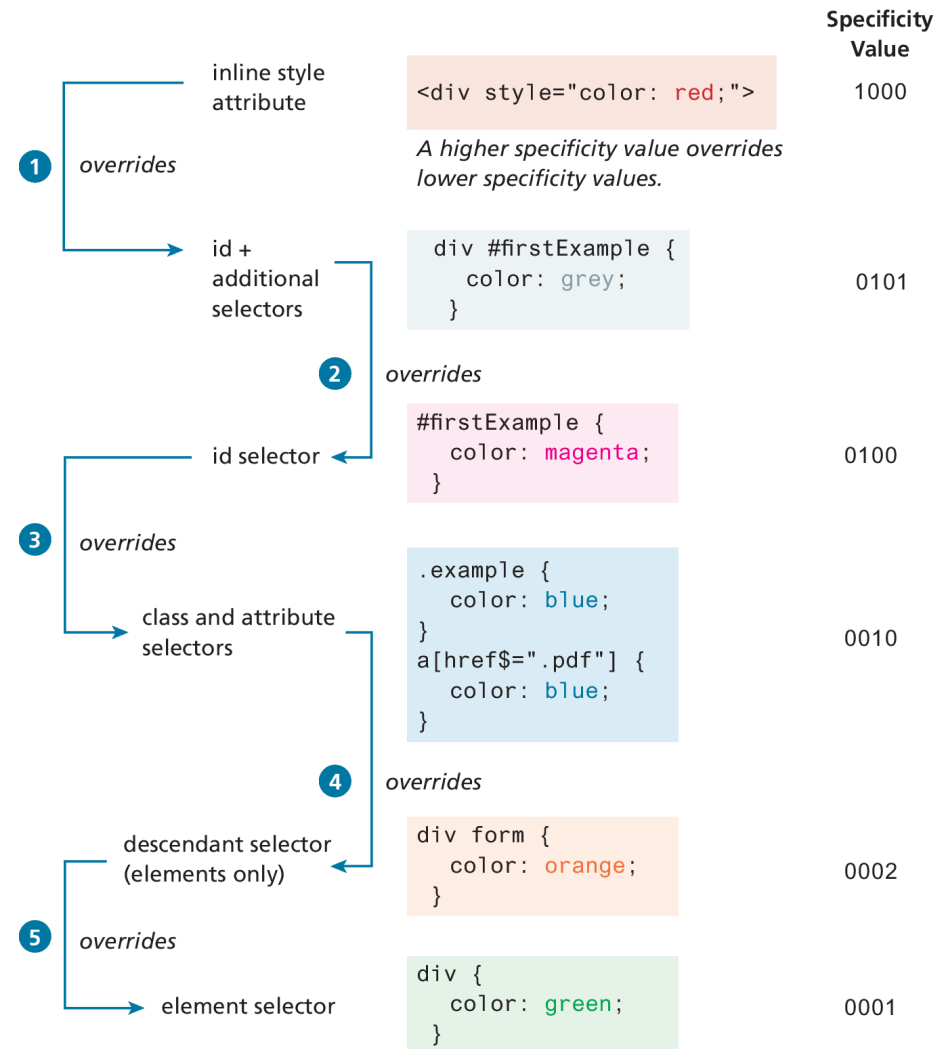
Specificity is how the browser determines which style rule takes precedence when more than one style rule could be applied.

- The more specific selector takes precedence
- Class selectors take precedence over element selectors, and id selectors take precedence over class selectors

Note: In figure 4.14 <body> color and font-weight properties are overridden by the more specific <div> and <p> selectors.



Simplified Specificity algorithm

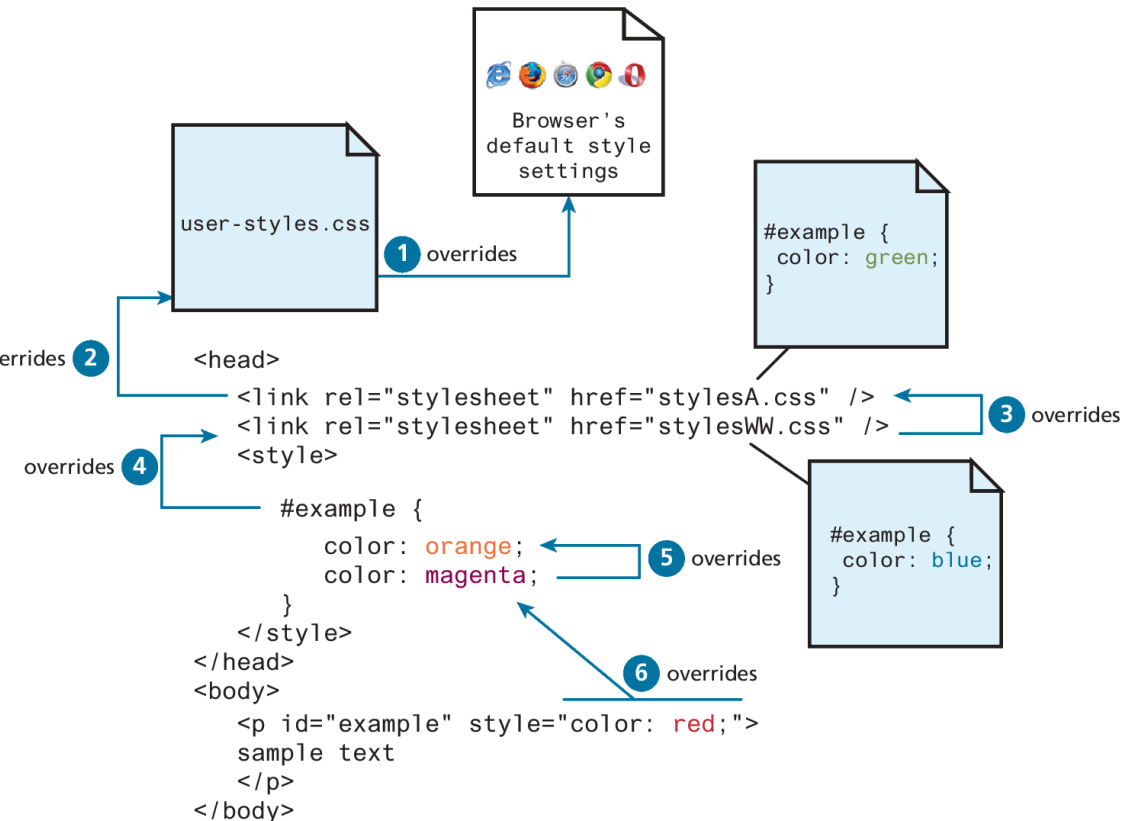


The Cascade: Location

Finally, when inheritance and specificity cannot determine style precedence, the principle of **location** will be used.

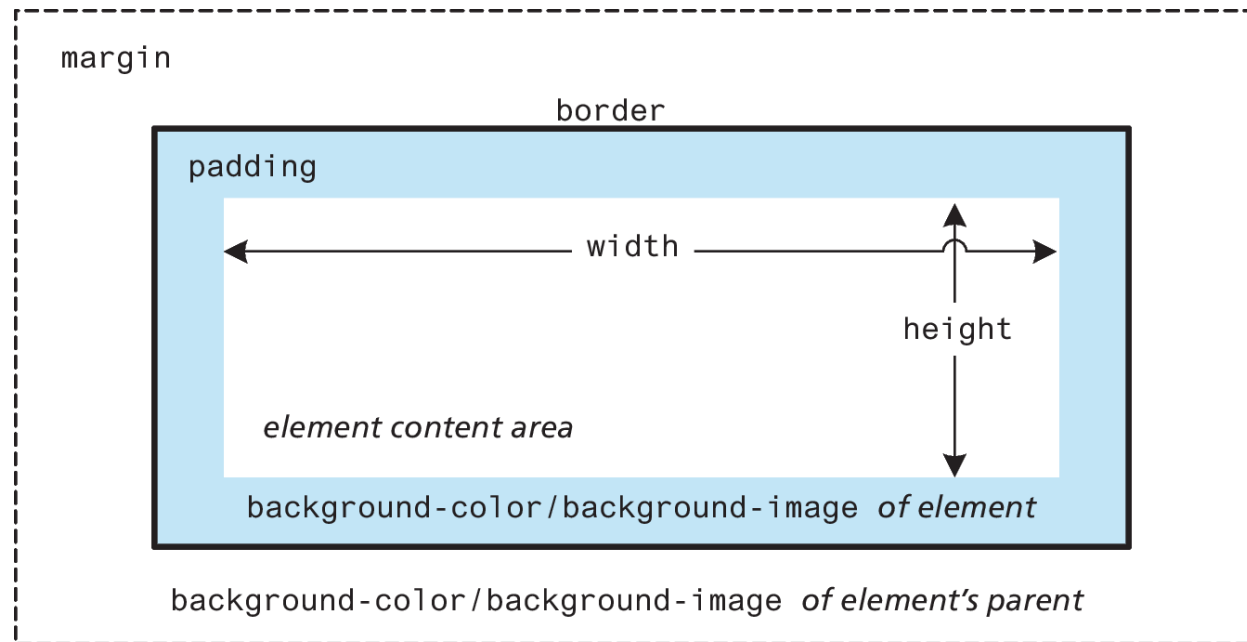
When rules have the same specificity, then the latest are given more weight.

For instance, an inline style will override one defined in an external author style sheet or an embedded style sheet.



The Box Model

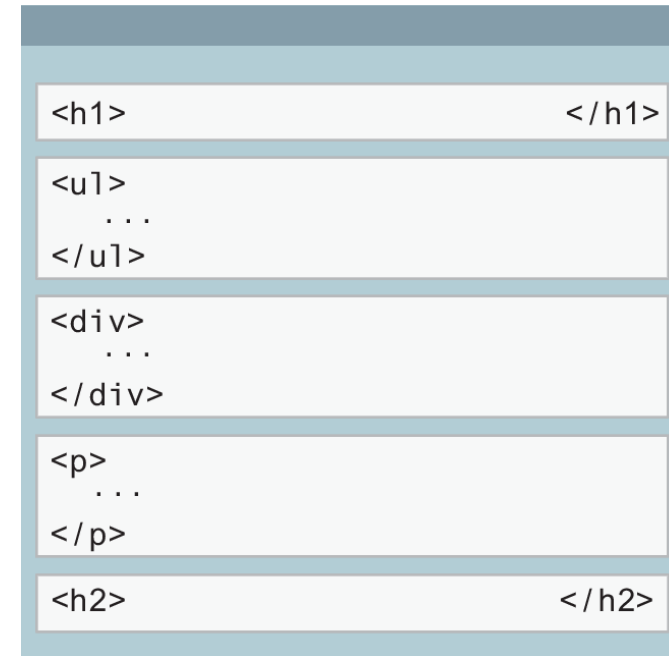
In CSS, all HTML elements exist within an **element box**. In order to become proficient with CSS, you must become familiar with the box model.



Block Elements

Block-level elements such as `<p>`, `<div>`, `<h2>`, `<ul>`, and `<table>` are each contained on their own line.

- without styling, two block-level elements can't exist on the same line
- Block-level elements use the normal CSS box model



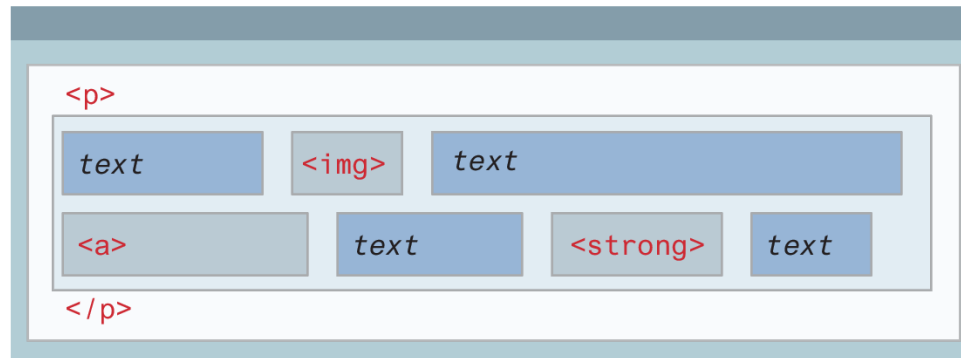
Inline Elements

Inline elements do not form their own blocks but instead are displayed within lines.

- Normal text in an HTML document is inline, as are elements such as `<em>`, `<a>`, `<img>`, and `<span>`.
- When there isn't enough space left on the line, the content moves to a new line

Inline Element Example

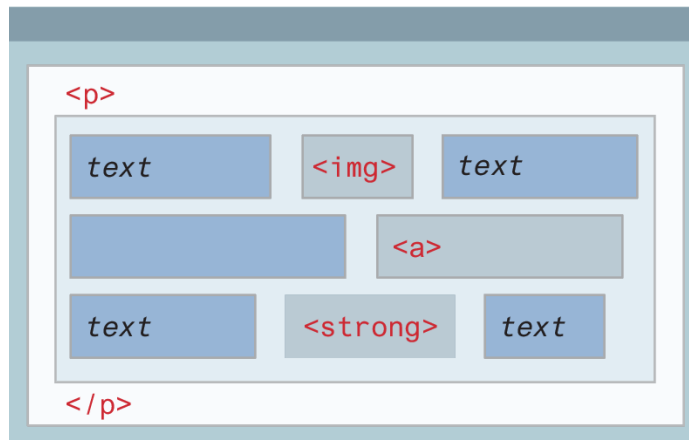
```
<p>  
This photo  of Conservatory Pond in  
<a href="http://www.centralpark.com">Central Park</a> was  
taken with a <strong>Canon EOS 30D</strong> camera.  
</p>
```



Inline content is laid out horizontally left to right within its container.

Once a line is filled with content, the next line will receive the remaining content, and so on.

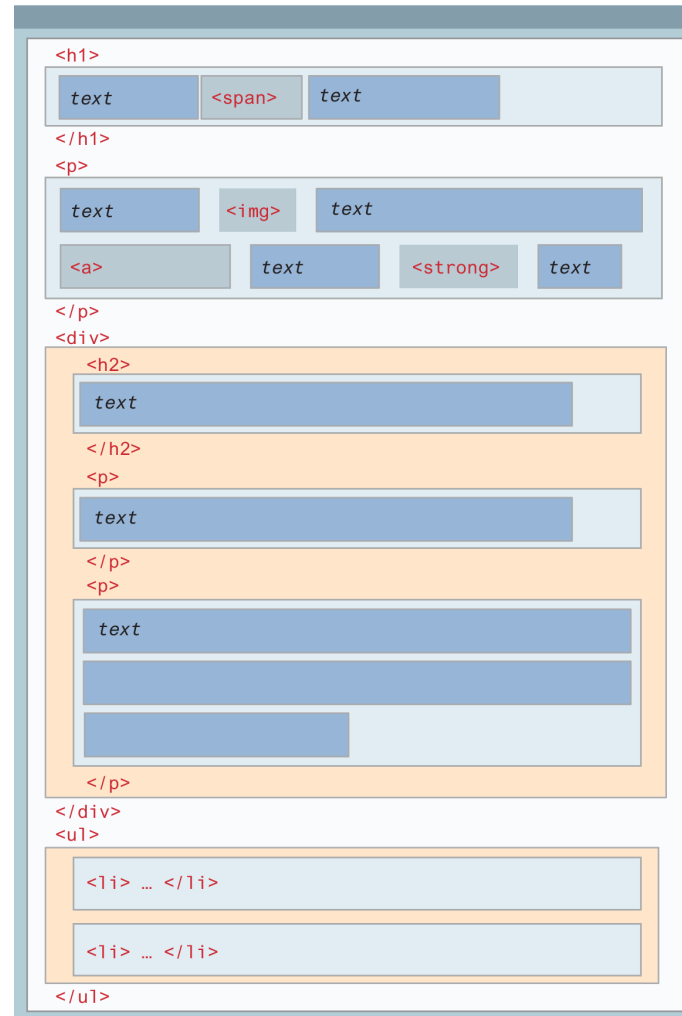
Here the content of this <p> element is displayed on two lines.



If the browser window resizes, then inline content will be “reflowed” based on the new width.

Here the content of this <p> element is now displayed on three lines.

Block and inline elements together



A document consists of block-level elements stacked from top to bottom.

Within a block, inline content is horizontally placed left to right.

Some block-level elements can contain other block-level elements (in this example, a `<div>` can contain other blocks).

In such a case, the block-level content inside the parent is stacked from top to bottom within the container (`<div>`).

Background

The background of an element fills an element out to its border

Background image properties include:

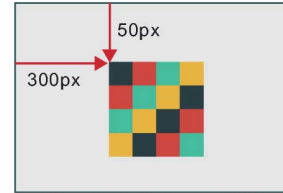
- **Background, background-attachment, background-color, background-image, background-origin, background-position, background-repeat, background-size**

Some of these properties work together, others do not.

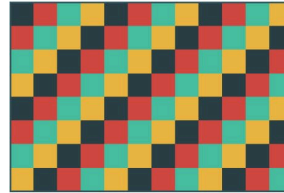
Background image examples



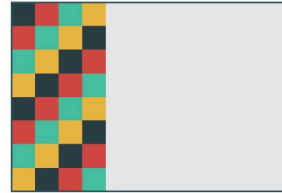
```
background-image: url(checkers.png);
background-repeat: no-repeat;
```



```
background-image: url(checkers.png) no-repeat;
background-position: 300px 50px;
```



```
background-repeat: repeat;
```



```
background-repeat: repeat-y;
```



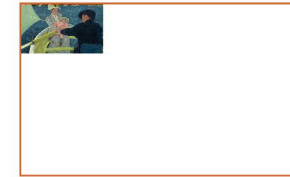
```
background-repeat: repeat-x;
```



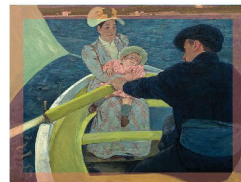
```
background-size: cover;
```



```
background-size: contain;
```



```
background-size: 200px 100px;
```



```
background-origin: border-box;
background-size: cover;
border-size: 5px;
border-color: rgba(242,112,90,0.5)
```



```
background-origin: padding-box;
```



```
background-origin: content-box;
padding: 10px;
```


Border and Shadow Examples



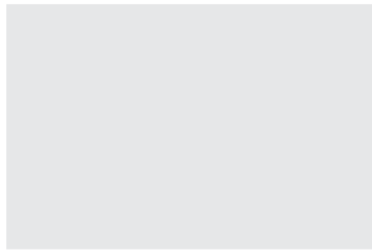
`border-style: solid;`



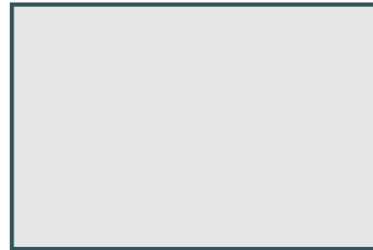
`border-style: dotted;`



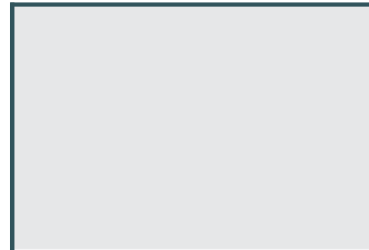
`border-style: dashed;`



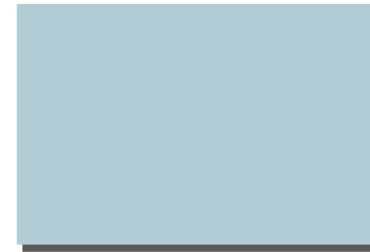
`border-width: 0;`



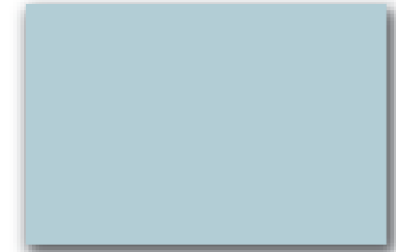
`border-width: 1px;`



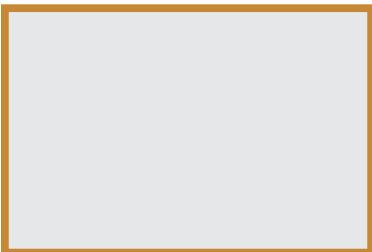
`border-width: 1px 2px 0 1px;`



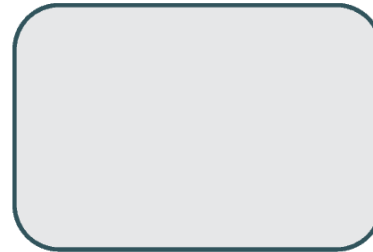
`box-shadow: 2px 2px gray;`



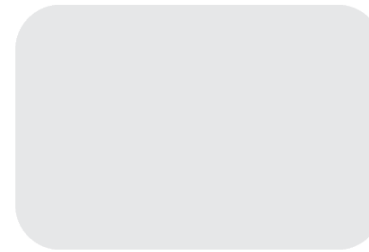
`box-shadow: 2px 2px 2px 2px gray;`



`border: solid 2px gold;`



`border-radius: 3px;`
`border-width: 1px;`



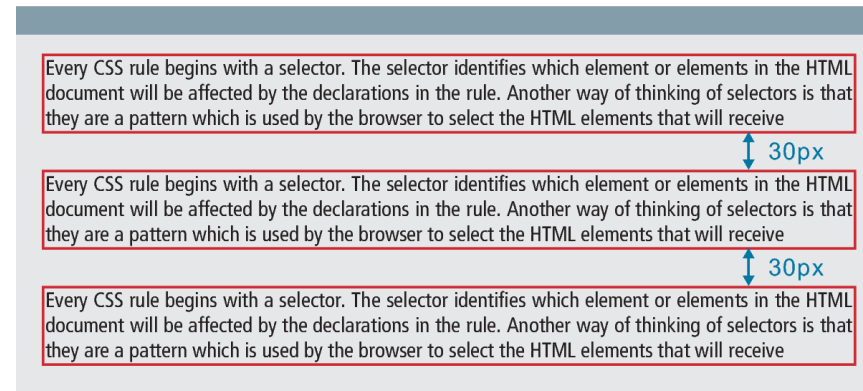
`border-radius: 3px;`
`border-width: 0;`

Margins and Padding

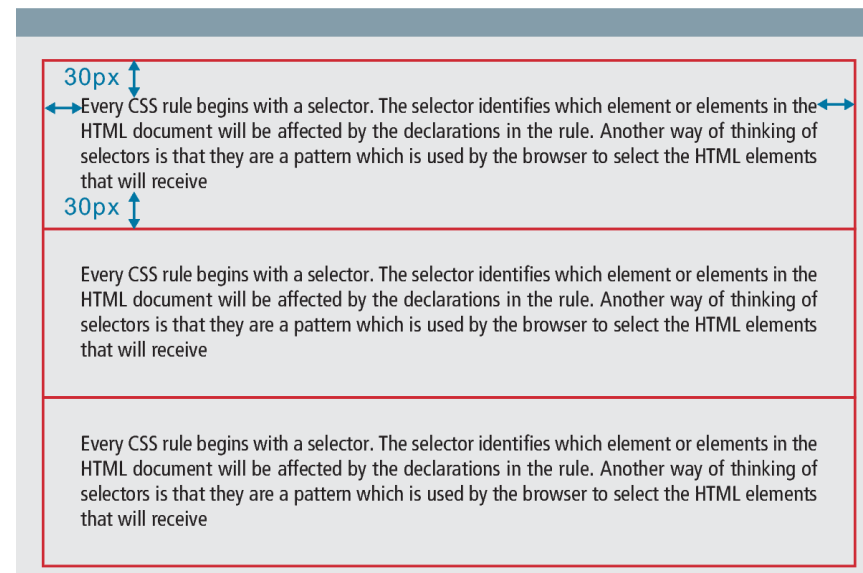
Margins add spacing around an element's content

Padding adds spacing within elements.

Borders divide the margin area from the padding area.



```
p {
  border: solid 1pt red;
  margin: 30px;
  padding: 0;
}
```



```
p {
  border: solid 1pt red;
  margin: 0;
  padding: 30px;
}
```

Collapsing margins

The W3C defines **collapsing margins** as:

- *In CSS, the adjoining margins of two or more boxes (which might or might not be siblings) can combine to form a single margin. Margins that combine this way are said to collapse, and the resulting combined margin is called a collapsed margin.*

What this means is that when the **vertical** margins of two elements touch, only the largest margin value of the elements will be displayed

Horizontal margins, on the other hand, **never** collapse.

Box Model: All or one

NOTE

With border, margin, and padding properties, it is possible to set the properties for one or more sides in a single property, or individually

```
border-top-color: red; /* sets just the top side */  
border-right-color: green; /* sets just the right side */  
border-bottom-color: yellow; /* sets just the bottom side */  
border-left-color: blue; /* sets just the left side */
```

Alternately, we can set all four sides to a single value via:

```
border-color: red; /* sets all four sides to red */
```

Or we can set all four sides to different values via:

```
border-color: red green orange blue;
```

Another shortcut is to use just two values; in this case the first value sets top and bottom, while the second sets the right and left.

```
border-color: red yellow; /* top+bottom=red, right+left=yellow */
```

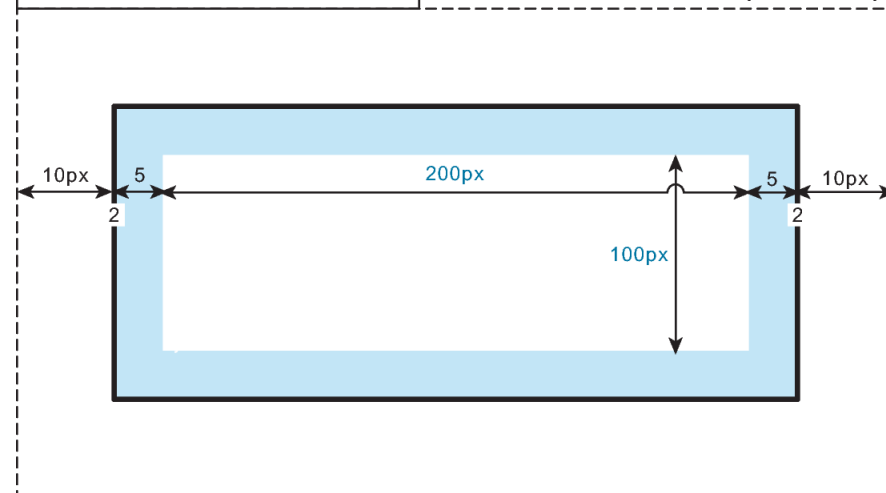


Box Dimensions

- Block-level elements have **width** and **height** properties.
- They also have a **min-width**, **min-height**, **max-width**, and **max-height** properties as well to help when the width or a height might be specified as a % of its parent container
- Since the width and the height only refer to the size of the content area, the **total size** of an element is equal to the size of its content area plus the sum of its padding, borders, and margins.

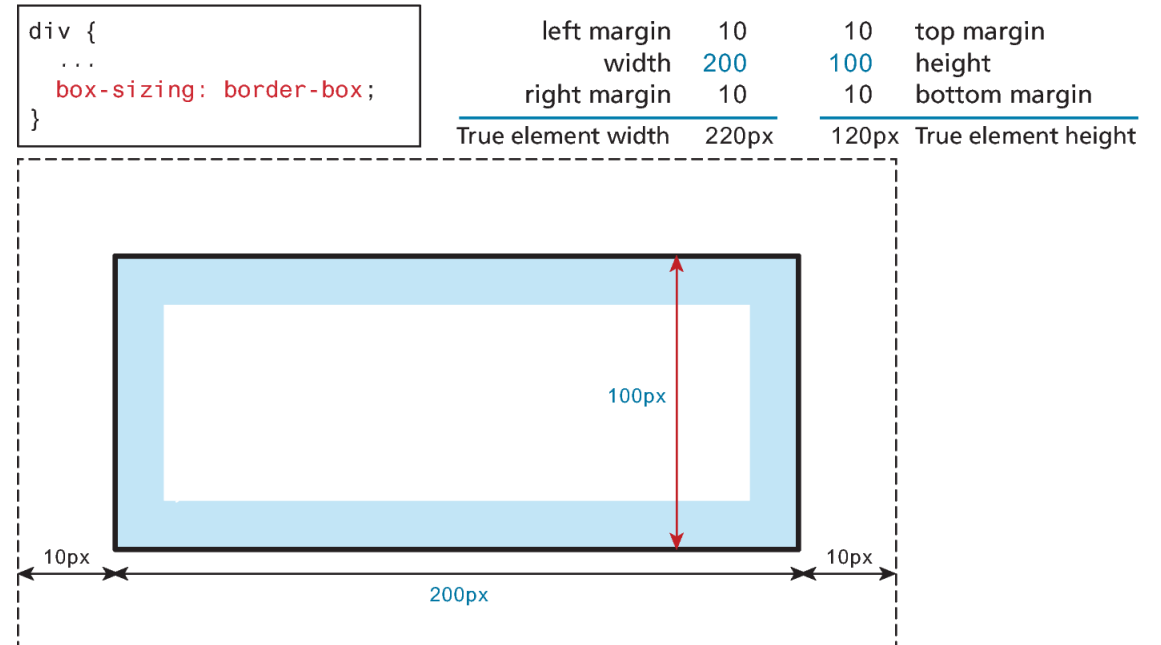
```
div {
  box-sizing: content-box;
  width: 200px;
  height: 100px;
  padding: 5px;
  margin: 10px;
  border: solid 2pt black;
}
```

left margin	10	10	top margin
left border	2	2	top border
left padding	5	5	top padding
width	200	100	height
right padding	5	5	bottom padding
right border	2	2	bottom border
right margin	10	10	bottom margin
True element width		234px	134px
		True element height	



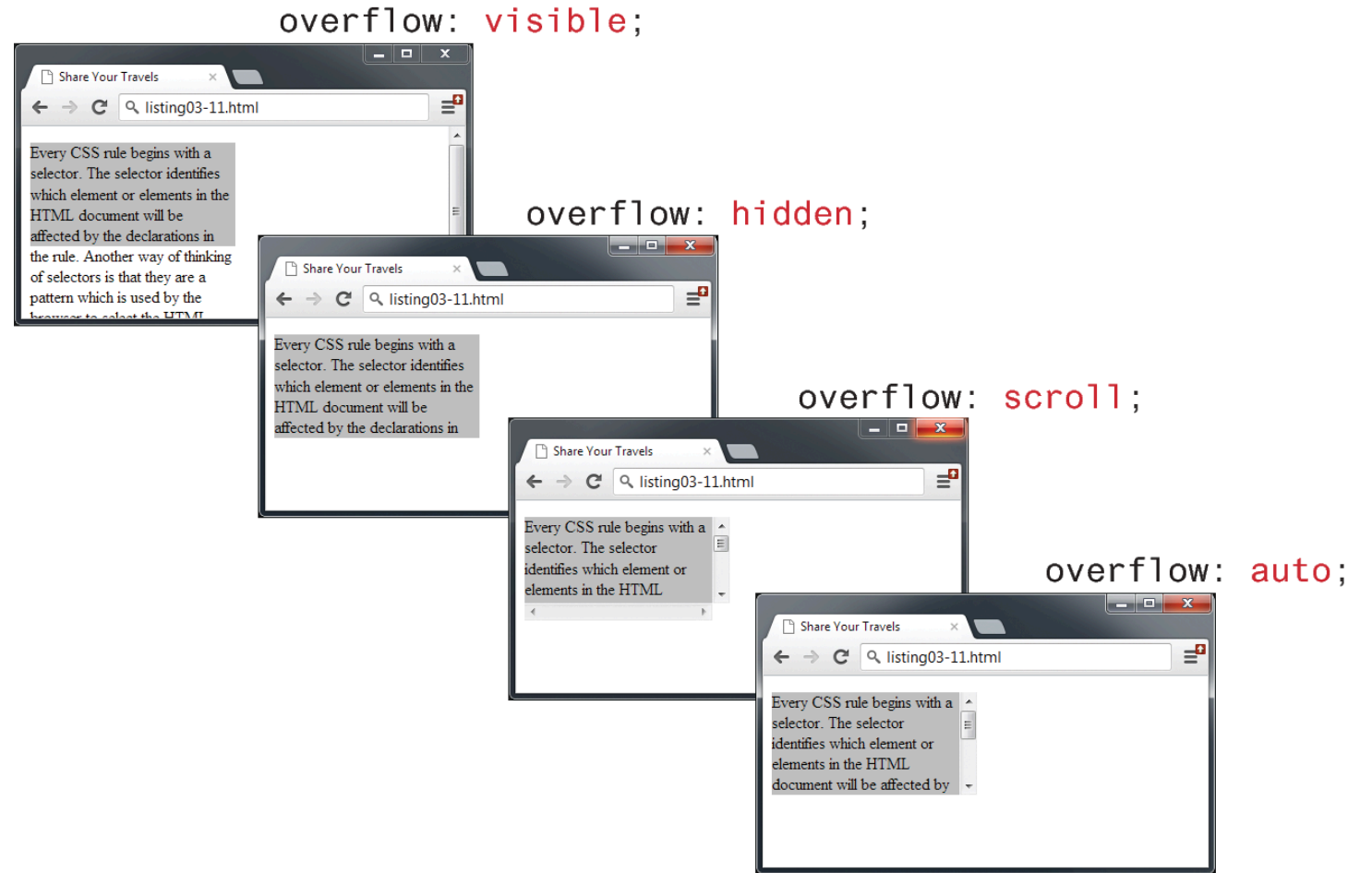
The border-box approach

To reduce complexity of adding margins, content and borders, you can use the newer border-box approach that is more intuitive.



Overflow Property

It is possible to control what happens with the content if the box's width and height are not large enough to display the content using the **overflow** property

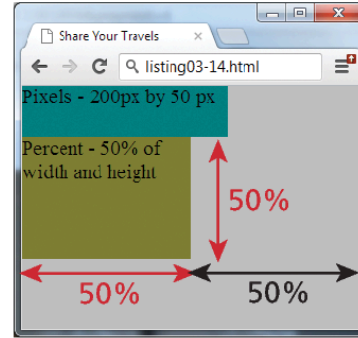


Box sizing via percent

```

<style>
  html,body {
    margin:0;
    width:100%;
    height:100%;
    background: silver;
  }
  .pixels {
    width:200px;
    height:50px;
    background: teal;
  }
  .percent {
    width:50%;
    height:50%;
    background: olive;
  }

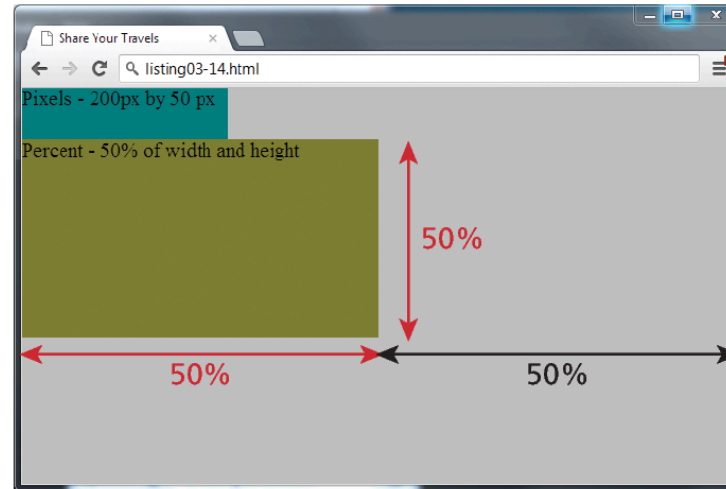
```



```

<body>
  <div class="pixels">
    Pixels - 200px by 50 px
  </div>
  <div class="percent">
    Percent - 50% of width and height
  </div>
</body>

```



CSS Text Styling

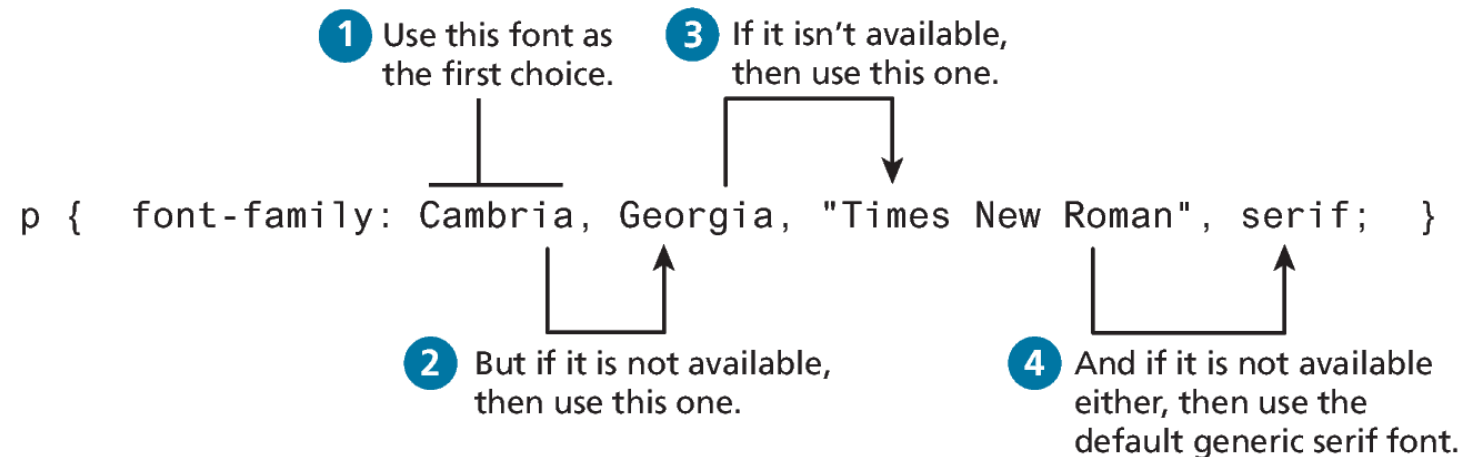
CSS provides two types of properties that affect text.

- **Font properties** that affect the font and its appearance.
- **Paragraph properties** that affect the text in a similar way no matter which font is being used
- Many of the most common font properties are shown in Table 4.9 and include
 - Font, font-family, font-style, font-size, font-variant, font-weight

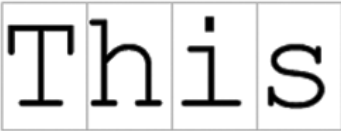
Font Family

Just because a given font is available on the web developer's computer does not mean it will be available for all users.

For this reason, it is conventional to supply a so-called **web font stack**— a series of alternate fonts to use in case the original font choice is not on the user's computer



Different font families

	Generic Font-Family Name		
This	serif		
This	sans-serif		Without ("sans") serif
This	monospace		In a monospace font, each letter has the same width.
This	cursive		
This	fantasy		
			In a regular, proportionally-spaced font, each letter has a variable width.

Decorative and cursive fonts
vary from system to system;
rarely used as a result.

Font Sizes

If we wish to create web layouts that work well on different devices, we should learn to use relative units such as **em units** or **percentages** for our font sizes

- it can quickly become difficult to calculate actual sizes when there are nested elements
 - For this reason, CSS3 now supports a new relative measure, the **rem** (for root em unit). This unit is always relative to the size of the root element (i.e., the `<html>` element).

To muddy the picture even more, some developers have begun to advocate again for using the pixel as the unit of measure in CSS since modern browsers provide built-in scaling/zooming that preserve layout...

@font-face

DIVE DEEPER

The @font-face selector that allows you to use a font even if it is not installed on the user's computer. Open source font sites such as Google Web Fonts (<https://fonts.google.com>) and Font Squirrel (<http://www.fontsquirrel.com/>) have promoted the use of @font-face

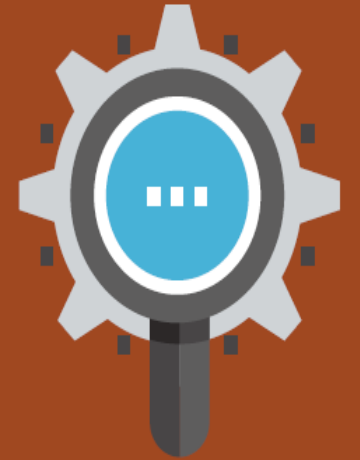
To use Droid Sans, you can add the following to your <head> section

```
<link href="https://fonts.googleapis.com/css?family=Droid+Sans" rel="stylesheet" type="text/css">
```

Or, add the following import inside one of your CSS files:

```
@import url(https://fonts.googleapis.com/css?family=Droid+Sans);
```

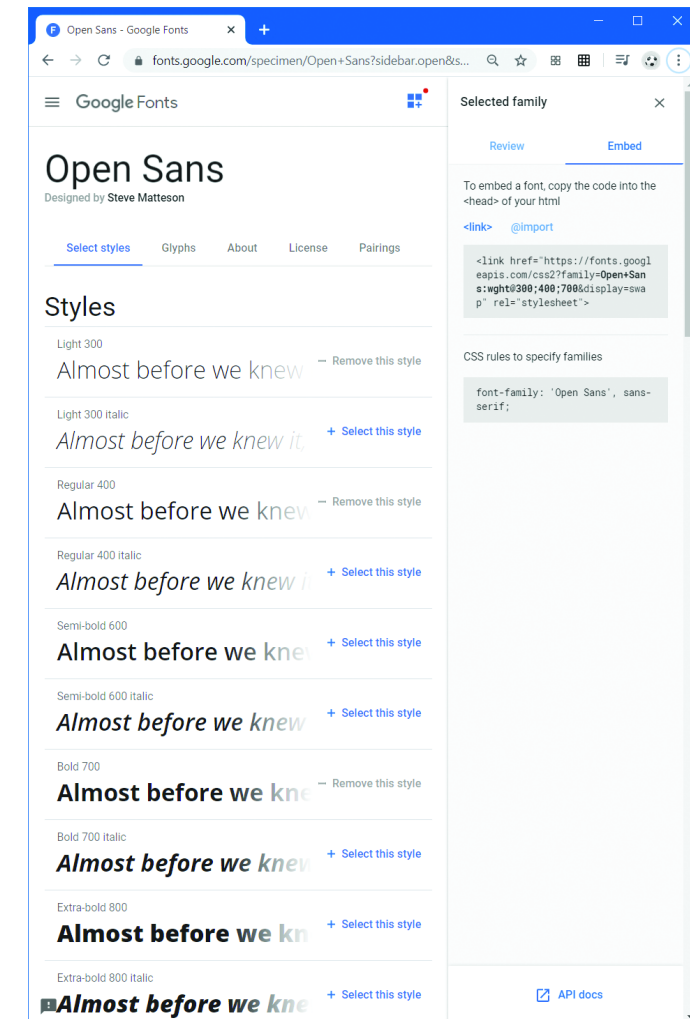
Or check out the longer-form @font-face selector (on p 170)



Font Weight

Google Fonts made web fonts available to the masses, before that **font-weight** was typically set to either normal or bold

- For instance, the popular Open Sans font has five different weights: **light**, **regular**, **semi-bold**, **bold**, and **extra bold**
- Within your CSS you specify which of these weights by using their numeric value, which typically ranges from 100 and 900, with larger numbers bolder than lower numbers



Paragraph Properties

Just as there are properties that affect the font in CSS, there is also a range of CSS properties that affect text independently of the font.

Table 2.10 describes the following paragraph properties in detail: **letter-spacing**, **line-height**, **list-style-image**, **list-style-type**, **text-align**, **text-decoration**, **text-direction**, **text-indent**, **text-shadow**, **text-transform**, **vertical-align** and **word-spacing**

Sample text properties

Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule.

`line-height: normal;`

Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule.

`line-height: 1.5;`

Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule.

`text-indent: 4em;`

Every CSS rule begins with a selector.

`text-align: left;`

Every CSS rule begins with a selector.

`text-align: center;`

Every CSS rule begins with a selector.

`text-align: right;`

Every CSS rule begins with a selector.

`letter-spacing: 3px;`



`vertical-align: top;`



`vertical-align: middle;`



`vertical-align: bottom;`

EVERY RULE BEGINS WITH A SELECTOR

`text-transform: uppercase;`

every rule begins with a selector

`text-transform: lowercase;`

selector

`text-decoration: underline;`

selector

`text-decoration: none;`

CSS Frameworks and Variables

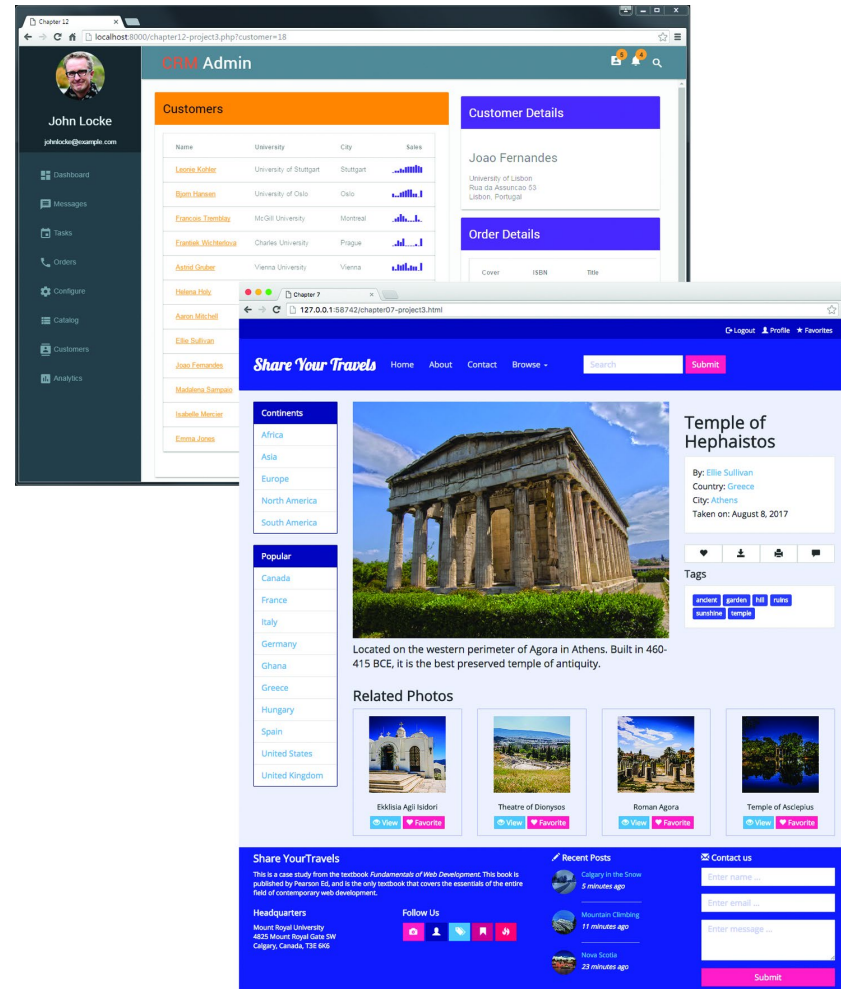
- This chapter and the exercises should make you reasonably confident about the **CSS box model** and **text formatting**
- Chapter 5 covers the CSS for working with forms and tables
- Chapter 7 covers layout, transitions, and animations as well
- If you are not feeling overwhelmed yet by CSS, you might be by the end of Chapter 7.
 - as an alternative to mastering CSS and getting an acceptable visual design, some developers instead use an already developed **CSS framework**

What is a CSS Framework?

A **CSS framework** is a set of CSS classes or other software tools that make it easier to use and work with CSS.

- Early CSS frameworks became popular chiefly as a way to more easily create complex grid-based layouts
- More sophisticated subsequent CSS Frameworks such as
 - Bootstrap (<https://getbootstrap.com/>) and
 - Foundation (<https://get.foundation/>) provide much more than a grid system; they provide a comprehensive set of predefined CSS classes, which makes it easier to construct a consistent and attractive web interface.
- Table 4.11 lists even more popular CSS Frameworks

Examples using Frameworks



CSS Variables

CSS Variables (also called custom properties) are a new feature that let you

- define variables (which must begin with a double hyphen) at the top of your CSS file usually within a special **:root** pseudo-class selector, and
- reference those variables as property values using the **var()** CSS function.

Duplication of colors, spacing, and borders, often happens numerous times within a single file (and rightly so). CSS variables address this issue without using a preprocessor (Chapter 7).

Example Part 1 (duplicate values)

```
header {  
  background-color: #431c5d;  
  color: #e05915;  
  padding: 4px;  
  box-shadow: 6px 5px 20px 1px rgba(0,0,0,0.22);  
  margin: 0;  
}  
header button {  
  background-color: #e05915;  
  border-radius: 5px;  
  border-color: #e6e9f0;  
  padding: 4px;  
  color: #e6e9f0;  
  font-size: 18px;  
  margin-top: 9px;  
}  
  
#results {  
  background-color: #431c5d;  
  font-size: 18px;  
  border-radius: 5px;  
  padding: 4px;  
  box-shadow: 6px 5px 20px 1px rgba(0,0,0,0.22);  
}
```

LISTING 4.8 Duplicate property values in CSS

Example Part 2 (CSS variables)

```
:root {
  --bg-color-main: #431c5d;
  --bg-color-secondary: #e05915;
  --fg-color-main: #e6e9f0;
  --radius-boxes: 5px;
  --padding-boxes: 4px;
  --font-size-default: 18px;
  --shadow-color: rgba(0,0,0,0.22);
  --dropshadow: 6px 5px 20px 1px var(--shadow-color);
}
header {
  background-color: var(--bg-color-main);
  color: var(--bg-color-secondary);
  padding: var(--padding-boxes);
  box-shadow: var(--dropshadow);
  margin: 0;
}
header button {
  background-color: var(--bg-color-secondary);
  border-radius: var(--radius-boxes);
  border-color: var(--fg-color-main);
  padding: var(--padding-boxes);
  color: var(--fg-color-main);
  font-size: var(--font-size-default);
  margin-top: calc( --font-size-default / 2 );
}
#results {
  background-color: var(--bg-color-main);
  font-size: var(--font-size-default);
  border-radius: var(--radius-boxes);
  padding: var(--padding-boxes);
  box-shadow: var(--dropshadow);
}
```

LISTING 4.9 Using CSS Variables

Key Terms

absolute units	combinators	element box	location	responsive design
attribute selector	contextual selector	element selectors	margin	selector
author-created style sheets	CSS	em units	Padding	specificity
block-level elements	CSS framework	embedded style sheets	percentages	style rules
box model	CSS variable	external style sheets	property	universal element
browser style sheets	CSS3 modules	generic font	property:value pair	selector
cascade	Declaration	grouped selector	pseudo-class selector	user style sheets
Cascading Style Sheets	declaration block	id selector	pseudo-element selector	vendor prefixes
class selector	descendant selector	inheritance	relative units	web font stack
collapsing margins	Document Object	inline styles	rem	x-height
	Model			

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar



Chapter 7

CSS 2: Layout

In this chapter you will learn . . .

- Approaches to CSS layout using CSS flexbox and grid models
- What responsive web design is and how to construct responsive designs
- How to use CSS3 filters, transitions, and animations
- What are CSS preprocessors

Older Approaches to CSS Layout

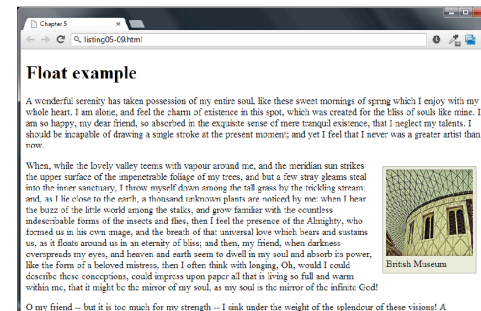
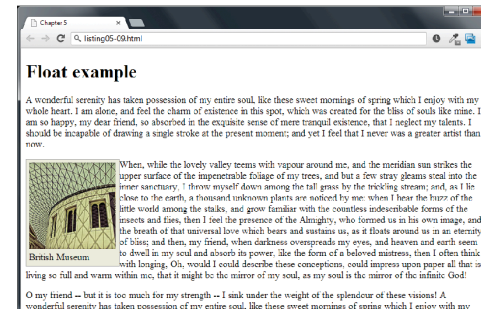
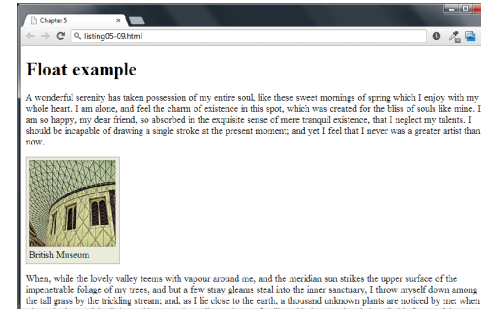
- The display property in CSS provides a mechanism for the developer to change an element to block, inline, or inline-block.
- For the first 20 years of CSS, designers had to “hack” together multi column layouts using **floats** and/or **positioning**. There may be times when you may have to support legacy CSS so it makes sense to learn the basics of floats and positioning.
- Newer approaches (flexbox and grid display modes) make columnar layouts much easier to implement.

Floating Elements

It is possible to displace an element out of its position in the normal flow via the CSS **float property**.

An element can be floated to the **left** or floated to the **right** and content is “reflowed” around the floated element

A floated block-level element should have a width specified



```
<h1>Float example</h1>
<p>A wonderful serenity has taken ...</p>
<figure>
  
  <figcaption>British Museum</figcaption>
</figure>
<p>When, while the lovely valley ...</p>
```

```
figure {
  border: 1pt solid #A8A8A8;
  background-color: #EDEDDE;
  margin: 0;
  padding: 5px;
  width: 150px;
}
```

Notice that a floated block-level element should have a width specified.

```
figure {
  ...
  width: 150px;
  float: left;
}
```

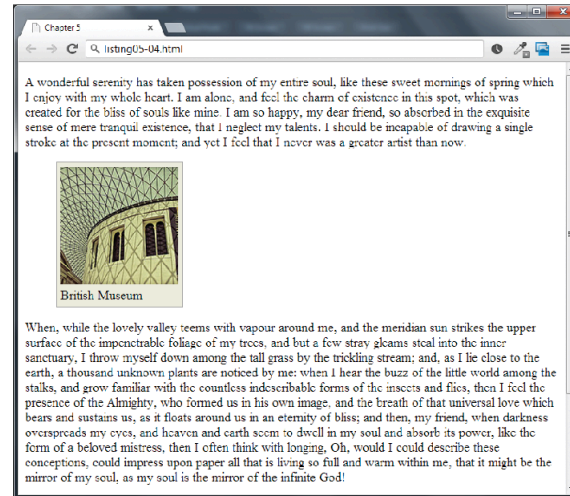
```
figure {
  ...
  width: 150px;
  float: right;
  margin: 10px;
}
```

Positioning Elements

The **position** property is used to specify the type of positioning. The **left**, **right**, **top**, and **bottom** properties are used to indicate the distance the element will move.

- **absolute** The element is removed from normal flow and positioned in relation to its nearest positioned ancestor.
- **fixed** The element is fixed in a specific position in the window even when the document is scrolled.
- **relative** The element is moved relative to where it would be in the normal flow.
- **static** The element is positioned according to the normal flow. **This is the default.**
- **sticky** The element is positioned in according to the normal flow, and then offset relative to its nearest scrolling ancestor. This is used to allow an item to scroll, and then stay fixed in position once its scroll position is reached.

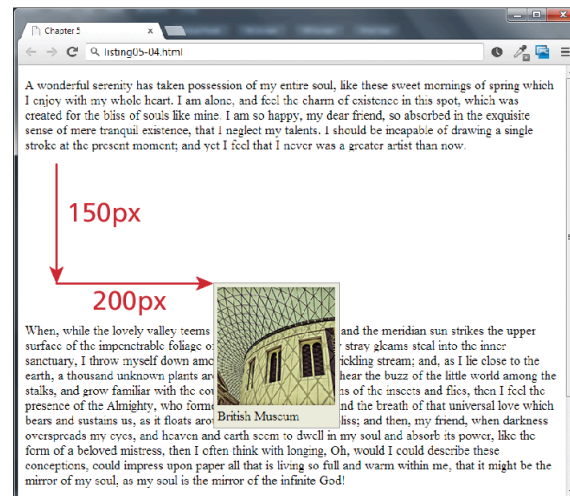
Relative Positioning



<p>A wonderful serenity has taken ... </p>

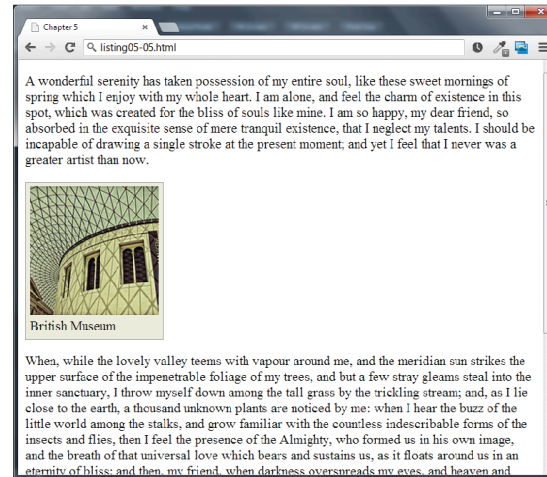
```
<figure>
  
  <figcaption>British Museum</figcaption>
</figure>
```

<p>When, while the lovely valley ...



```
figure {
  border: 1pt solid #A8A8A8;
  background-color: #EDEDDE;
  padding: 5px;
  width: 150px;
  position: relative;
  top: 150px;
  left: 200px;
}
```

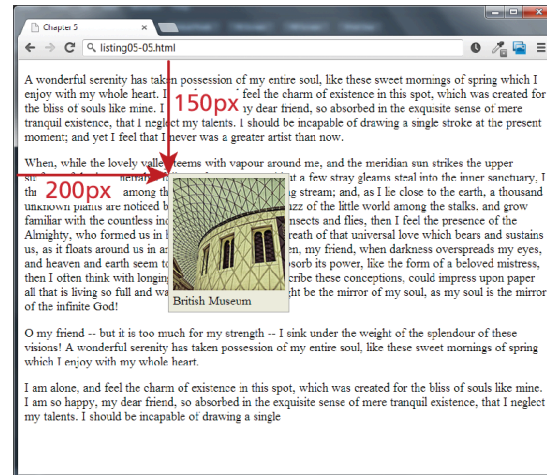
Absolute positioning



<p>A wonderful serenity has taken possession of my ...

```
<figure>
  
  <figcaption>British Museum</figcaption>
</figure>
```

<p>When, while the lovely valley ...



```
figure {
  margin: 0;
  border: 1pt solid #A8A8A8;
  background-color: #EDEDDE;
  padding: 5px;
  width: 150px;
  position: absolute;
  top: 150px;
  left: 200px;
}
```

Overlapping and Hiding Elements

One of the more common design tasks with CSS is to place two elements on top of each other, or to selectively hide and display elements.

In such a case, relative positioning is used to create the **positioning context** for a subsequent absolute positioning move.

The diagram illustrates two browser window states to demonstrate CSS positioning. The top window shows a default layout with an image of the British Museum and a caption below it. The bottom window shows the same layout with the caption moved to overlap the image. Red arrows in the bottom window indicate the movement: 20px left and 130px top from the original caption position.

HTML Code:

```
<figure>
  
  <figcaption>British Museum</figcaption>
</figure>
<p>When, while the lovely valley teems with vapour around me
```

CSS Code:

```
figure {
  ...
  position: relative;
}
figcaption {
  ...
  position: absolute;
  top: 130px;
  left: 20px;
}
```

Annotations:

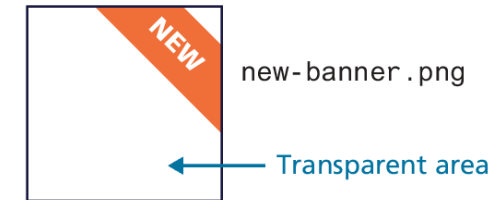
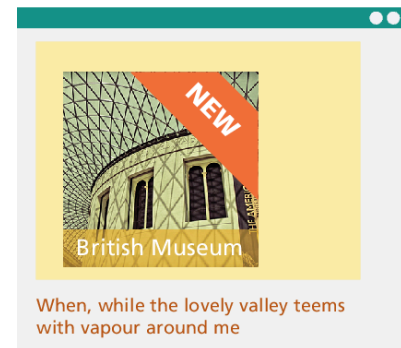
- position: relative;** This creates the positioning context.
- position: absolute;** This does the actual move.

Overlapping and Hiding Elements (ii)

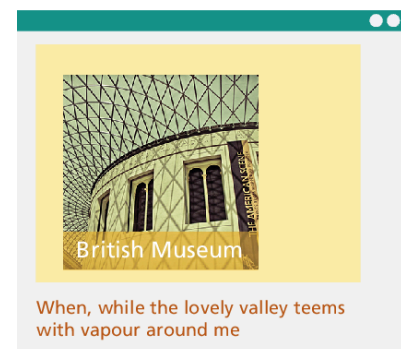
Consider an image that is the same size as the underlying one is placed on top of the other image using absolute positioning.

You can hide this image using the **display** property

```
<figure>
  
  <figcaption>British Museum</figcaption>
  
</figure>
```



```
.overlaid {
  position: absolute;
  top: 10px;
  left: 10px;
}
```



```
.hide {
  display: none;
}
```

This is the preferred way to hide: by adding this additional class to the element.

```

```


Flexbox Layout

← float



Fall in Calgary

Nunc nec fermentum dolor. Duis at iaculis turpis. Sed rutrum elit ac egestas dapibus. Duis nec consequat enim.

Mauris porta arcu id magna adipiscing lacinia at congue lacus. Vivamus blandit quam quis tincidunt egestas. Etiam posuere lectus sed sapien malesuada molestie.

Phasellus vel felis purus. Aliquam consequat pellentesque dui, non mollis erat dictum sit amet. Curabitur non quam dictum, consectetur arcu in, vehicula justo.

margin-left
= image size + margin-right

```
.media-image {
  float: left;
  margin-right: 10px;
}
.media-body {
  margin-left: 160px;
}
```

```
<div class="media">
  
  <div class="media-body">
    <h2>Fall in Calgary</h2>
    <p>Nunc nec fermentum dolor...</p>
    <p>Mauris porta arcu id...</p>
    <p>Phasellus vel felis purus...</p>
  </div>
</div>
```

Prior to flexbox, one would create such a layout within a container using floats plus margins. The problem with this approach is that margins needed to be in pixels and had to exactly match image size. If image size changed (or you wanted same kind of style elsewhere), you had to modify the style.



Fall in Calgary

Nunc nec fermentum dolor. Duis at iaculis turpis. Sed rutrum elit ac egestas dapibus. Duis nec consequat enim.

Mauris porta arcu id magna adipiscing lacinia at congue lacus. Vivamus blandit quam quis tincidunt egestas. Etiam posuere lectus sed sapien malesuada molestie.

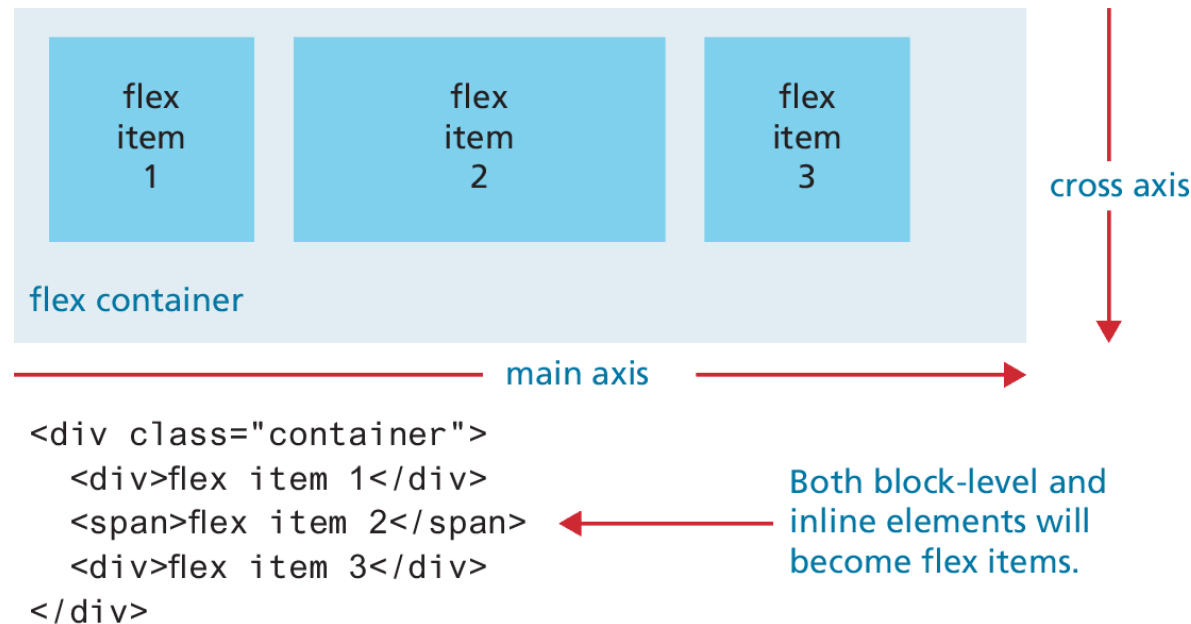
Phasellus vel felis purus. Aliquam consequat pellentesque dui, non mollis erat dictum sit amet. Curabitur non quam dictum, consectetur arcu in, vehicula justo.

```
.media {
  display: flex;
  align-items: flex-start;
}
.media-image {
  margin-right: 1em;
}
```

Using flexbox, we now have a much more generalized (and thus reusable) style.

Flex containers and items

There are two places in which you will be assigning flexbox properties: the **flex container** and the **flex items** within the container.



The flexbox container properties

`display: flex` Necessary to use flexbox

`flex-direction: row`



Specifies the direction of the main axis.
The default is `row`.

`flex-direction: row-reverse`

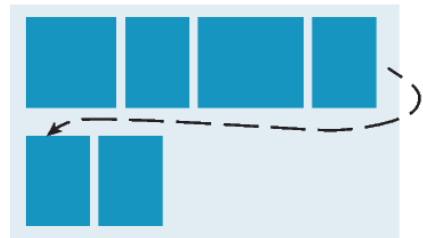


`flex-direction: column`

`flex-direction: column-reverse`



`flex-wrap: wrap`



`flex-wrap: nowrap`



Indicates whether new lines should be created if flex container can't contain all the flex items.

The flexbox container properties (ii)

`justify-content: flex-start`

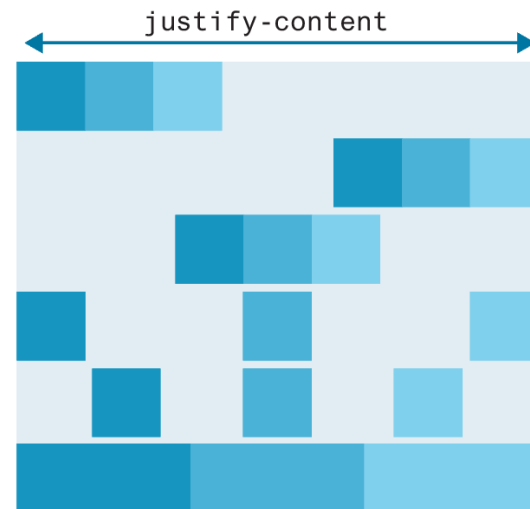
`justify-content: flex-end`

`justify-content: center`

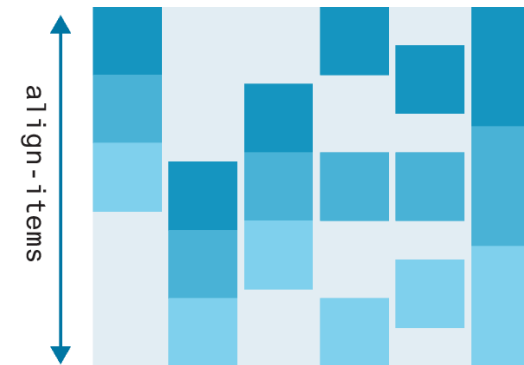
`justify-content: space-between`

`justify-content: space-around`

`justify-content: stretch`



Controls flex item alignment along main axis.



Controls flex item alignment along cross axis (i.e., wrapped items).

`align-items: stretch`
`align-items: space-around`
`align-items: space-between`
`align-items: center`
`align-items: flex-end`
`align-items: flex-start`

The flexbox item (child) properties

flex-basis: **auto**



The flex-basis property determines the initial size of the flex item before the remaining space is distributed.

The default auto value means that the size is determined by the width and height.

flex-basis: **200px**



You can specify a width using px, %, or other measurement units.

flex-grow: **2**



width = n

width = $n \times 2$

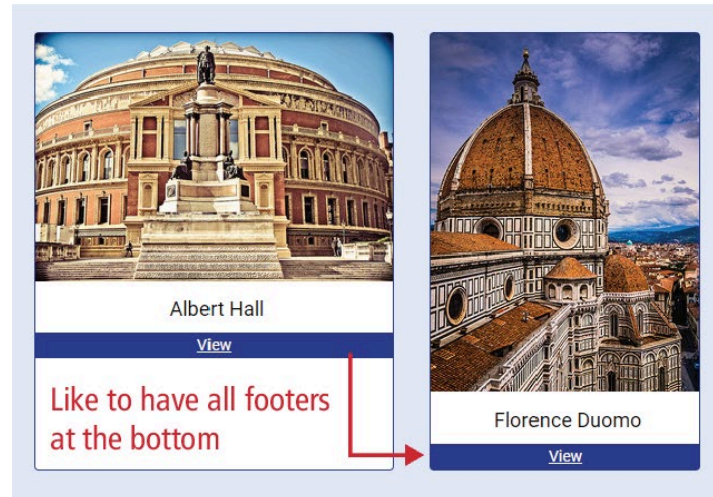
Defines the growth factor of an element relative to the other items.

flex-shrink: **2**



Defines how much an item will shrink when not enough space in container.

Flexbox Cards



```
<div class="card">
  <div class="content">
    
    <h3>Albert Hall</h3>
  </div>
  <footer>
    <a href="#">View</a>
  </footer>
</div>
```



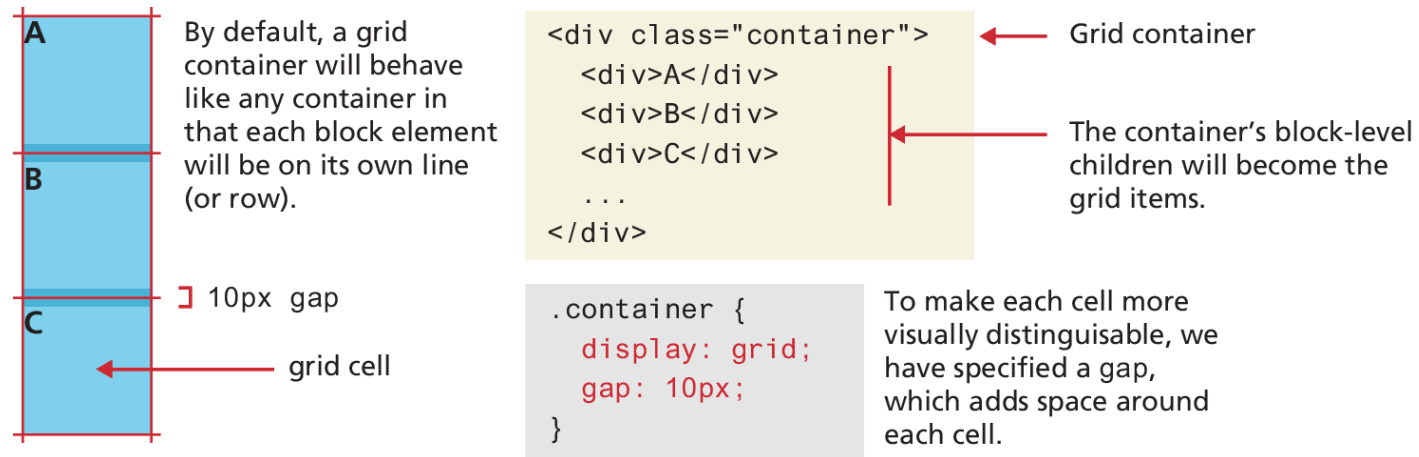
```
.card {
  ...
  display: flex;
  flex-direction: column;
}

.card .content {
  flex: 1 1 auto;
}
```

Grid Layout

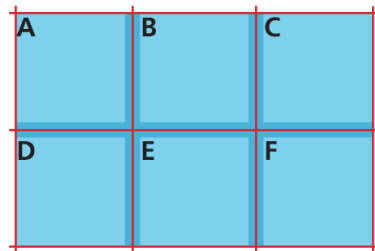
Grid layout is adjustable, powerful, and, compared to floats, positioning, and even flexbox, is relatively easy to learn and use!

- Each blocklevel child in a parent container whose **display** property is set to **grid** will be automatically placed into a grid cell



Specifying Grid Structure

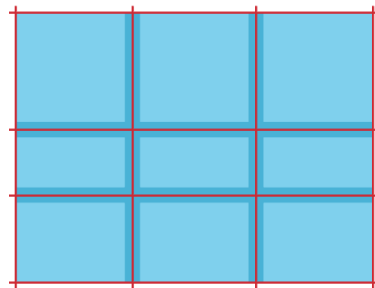
grid-template-columns is used for adding columns by specifying each column's width using the **fr** unit.



```
.container {  
  display: grid;  
  gap: 10px;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

This makes each column equal to an equal fraction of the available space.

You can specify the number of columns per row/line via the grid-template-column property.



```
.container {  
  ...  
  height: 300px;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 80px 30px 50px;  
}
```

While you often do not need to set a row height, you can make rows taller than their actual content.

Specifying column widths

Column widths can be specified

The CSS **repeat()** function provides a way to specify repeating patterns of columns.

```
grid-template-columns: 70px 70px 45px;
```



Each column can have its own unique width value, in px, %, em, etc.

```
grid-template-columns: 50px auto 50px;
```



An auto value indicates the width will fill the remaining space.

```
grid-template-columns: repeat(3, 1fr);
```



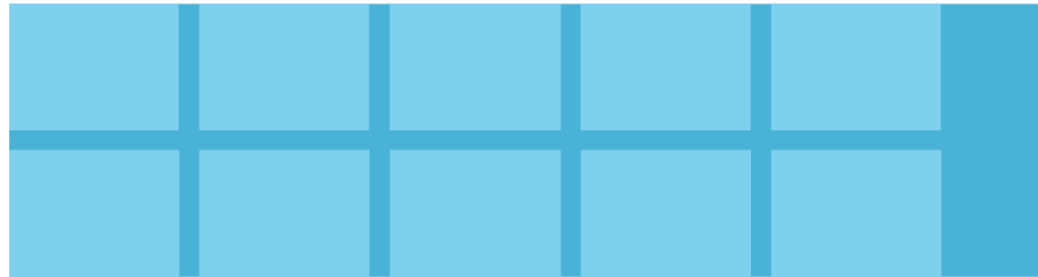
The repeat function can be used to defining a repeating pattern.

```
grid-template-columns: repeat(2, 30px 50px) 70px;
```



30px 50px 30px 50px 70px

Specifying column widths (ii)



```
grid-template-columns: repeat(auto-fill, minmax(100px, 1fr));
```

Fills the row with as many columns as can fit into the container space.

The min column size is 100px and the max is whatever size is necessary to allow each column to be equal sized.



```
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
```

Fits the columns into space by expanding the column width into the available container space.

Contrasting grid approaches

<!-- Bootstrap 4 Approach -->

```
<div class="container">
  <div class="row">
    <div class="col"><img src=1.gif /></div>
    <div class="col"><img src=2.gif /></div>
    <div class="col"><img src=3.gif /></div>
  </div>
  <div class="row">
    <div class="col"><img src=4.gif /></div>
    <div class="col"><img src=5.gif /></div>
    <div class="col"><img src=6.gif /></div>
  </div>
</div>
```

<!-- CSS Grid Approach -->

```
<div class="container">
  <img src=1.gif />
  <img src=2.gif />
  <img src=3.gif />
  <img src=4.gif />
  <img src=5.gif />
  <img src=6.gif />
</div>
```

<!-- CSS for grid approach -->

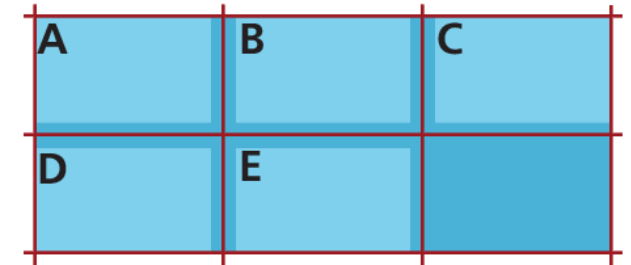
```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
}
.container img { display: block; }
```

LISTING 7.1 Comparing Bootstrap grid with CSS Grid

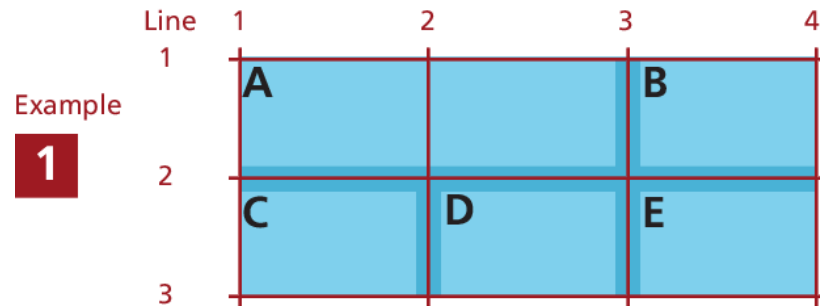
Explicit Grid Placement Example 1

```
<div class="container">
  <div class="a">A</div>
  <div class="b">B</div>
  <div class="c">C</div>
  <div class="d">D</div>
  <div class="e">E</div>
</div>
```

```
.container {
  display: grid;
  gap: 10px;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: repeat(2, 200px);
}
```



With implicit layout, grid items are placed automatically.



```
.a {
  grid-column-start: 1;
  grid-column-end: 3;
}
```

The start and end numbers refer to the line number not the column number.

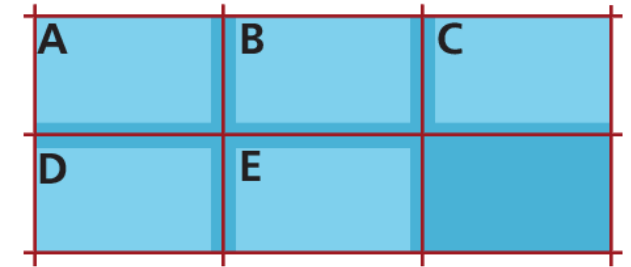
The same effect also possible using either of the following:

```
grid-column: 1 / 3;
grid-column: 1 / span 2;
```

Explicit Grid Placement Example 2

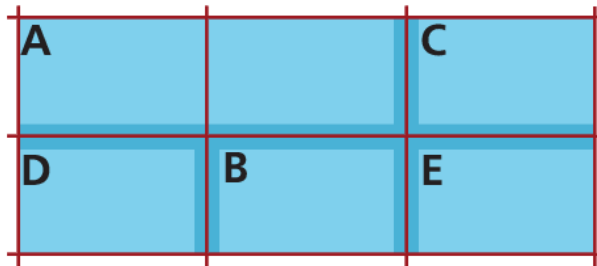
```
<div class="container">  
  <div class="a">A</div>  
  <div class="b">B</div>  
  <div class="c">C</div>  
  <div class="d">D</div>  
  <div class="e">E</div>  
</div>
```

```
.container {  
  display: grid;  
  gap: 10px;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: repeat(2, 200px);  
}
```



With implicit layout, grid items are placed automatically.

2

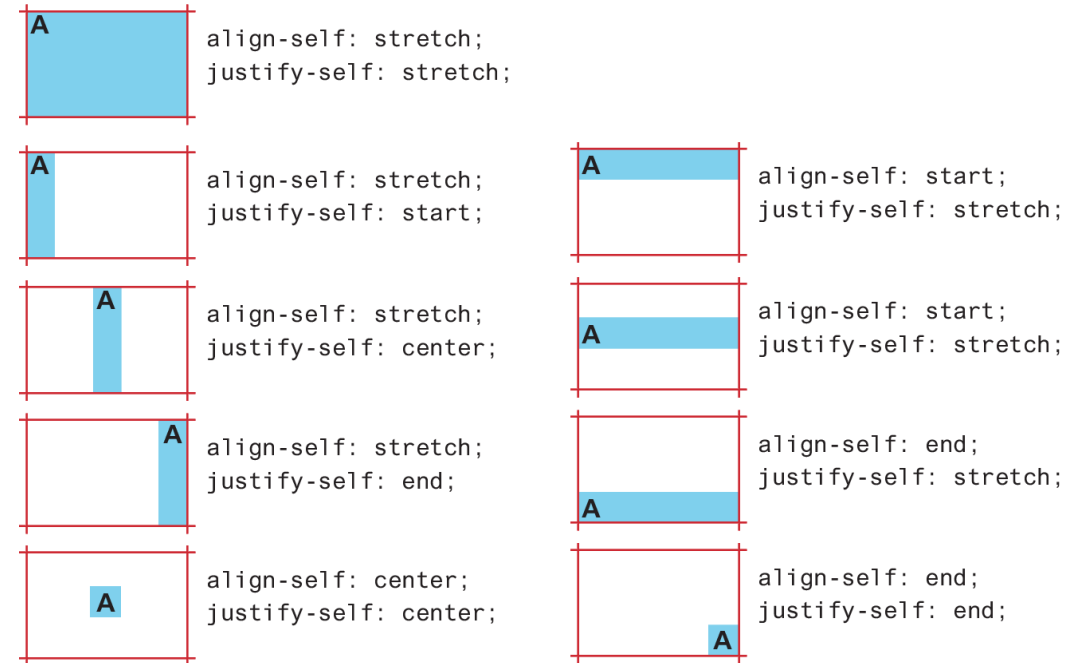


```
.b {  
  grid-row: 2;  
  grid-column: 2;  
}
```

Grid cells can be placed into any row and column.

Cell properties

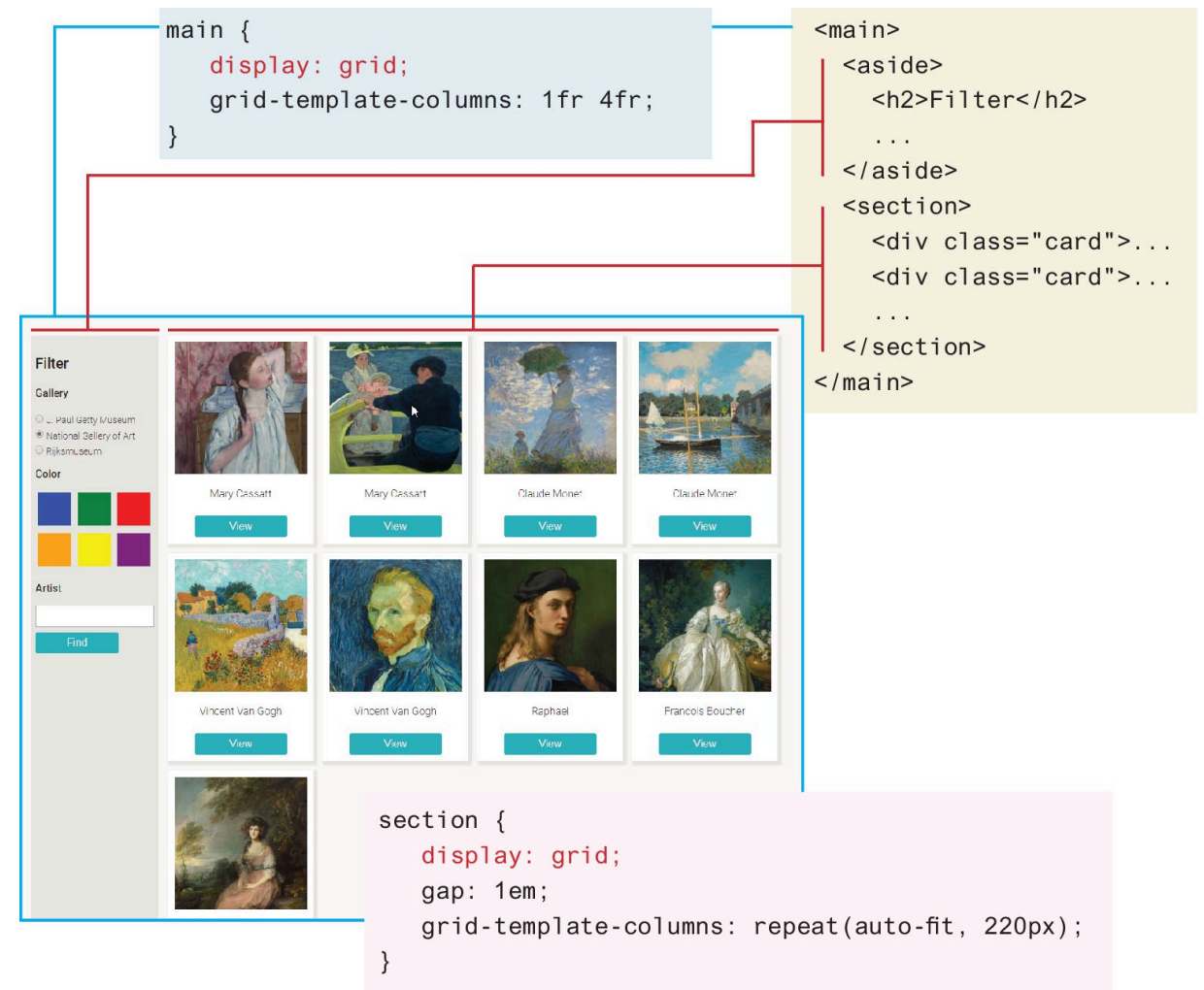
- **align-self** and **justify-self** control the cell content's horizontal and vertical alignment within its grid container.



- You can similarly control cell alignment within a grid container using **align-items** and **justify-items**

Nested Grid

- **align-self and justify-self**
control the cell content's
horizontal and vertical alignment
within its grid container.



Grid Areas

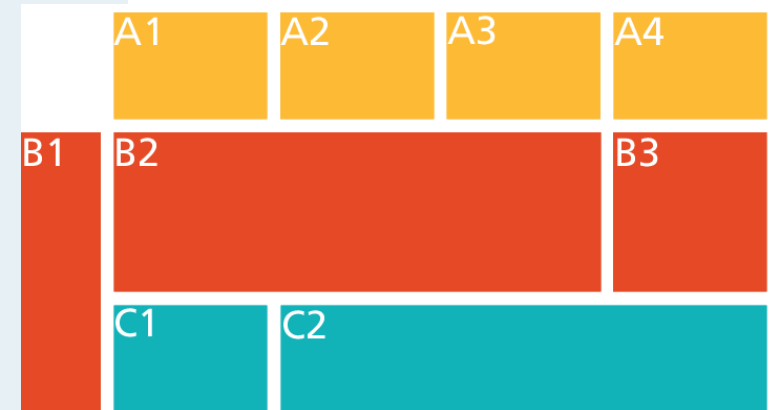
```

<style>
.container {
  grid-gap: 10px;
  display: grid;
  grid-template-rows: 100px 150px 100px;
  grid-template-columns: 75px 1fr 1fr 1fr 1fr;
  grid-template-areas: ". a1 a2 a3 a4"
                      "b1 b2 b2 b2 b3"
                      "b1 c1 c2 c2 c2";
}
.a1 { grid-area: a1; }
.a2 { grid-area: a2; }
.a3 { grid-area: a3; }
.a4 { grid-area: a4; }
.b1 { grid-area: b1; }
.b2 { grid-area: b2; }
.b3 { grid-area: b3; }

.c1 { grid-area: c1; }
.c2 { grid-area: c2; }
</style>

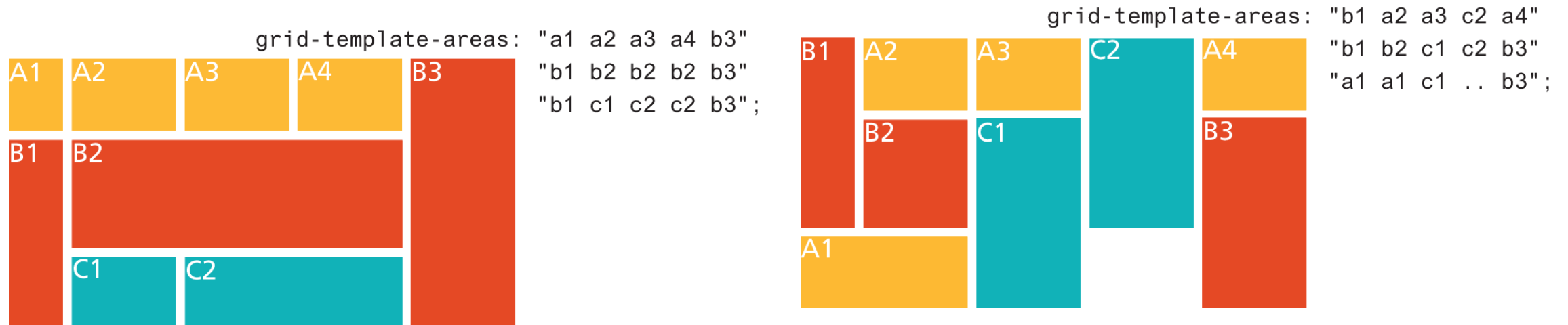
<section class="container">
  <div class="yellow a1">A1</div>
  <div class="yellow a2">A2</div>
  <div class="yellow a3">A3</div>
  <div class="yellow a4">A4</div>
  <div class="orange b1">B1</div>
  <div class="orange b2">B2</div>
  <div class="orange b3">B3</div>
  <div class="cyan c1">C1</div>
  <div class="cyan c2">C2</div>
</section>

```



LISTING 7.2 Using grid areas

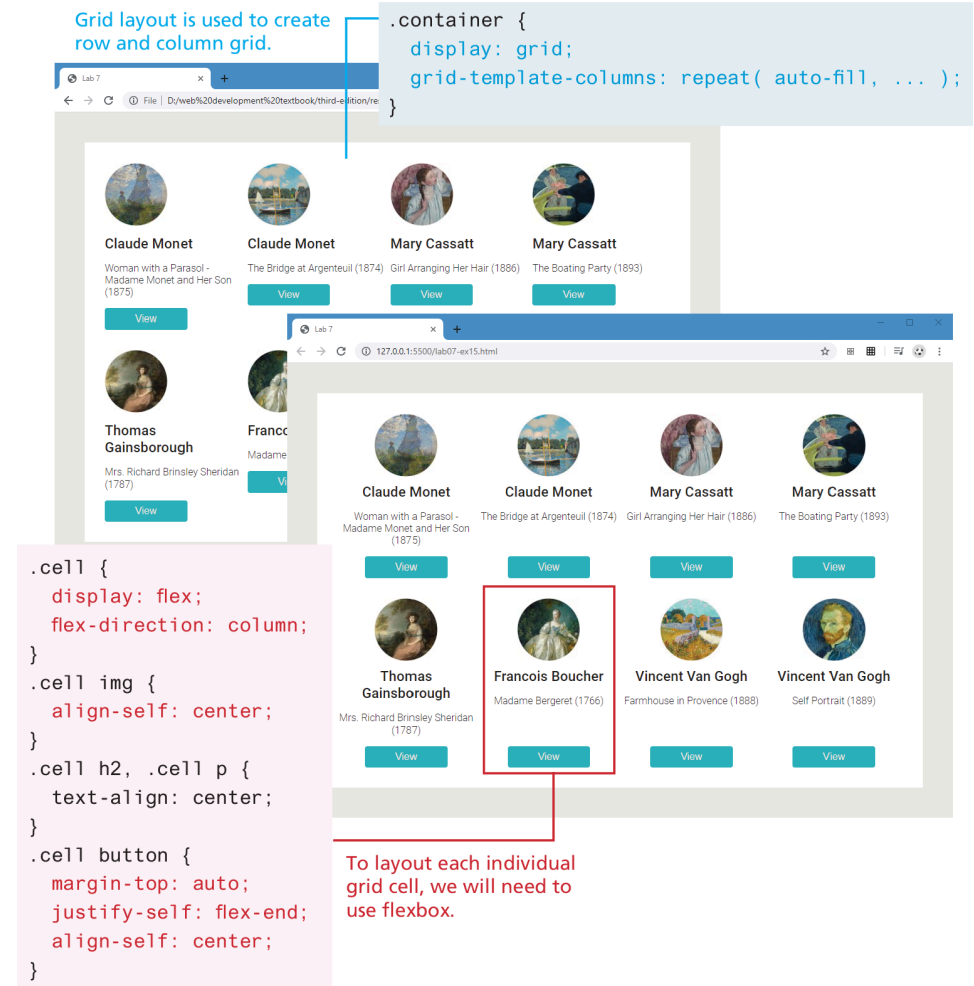
Grid Areas (ii)



LISTING 7.2 Using grid areas

Grid and Flexbox Together

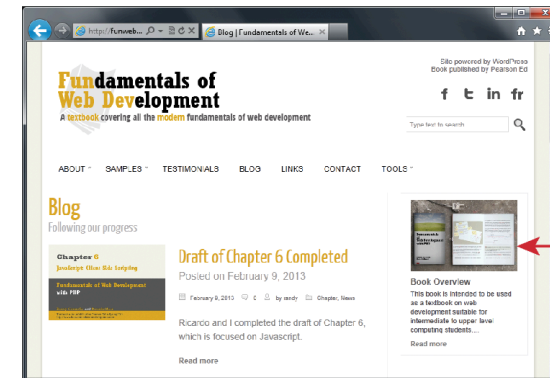
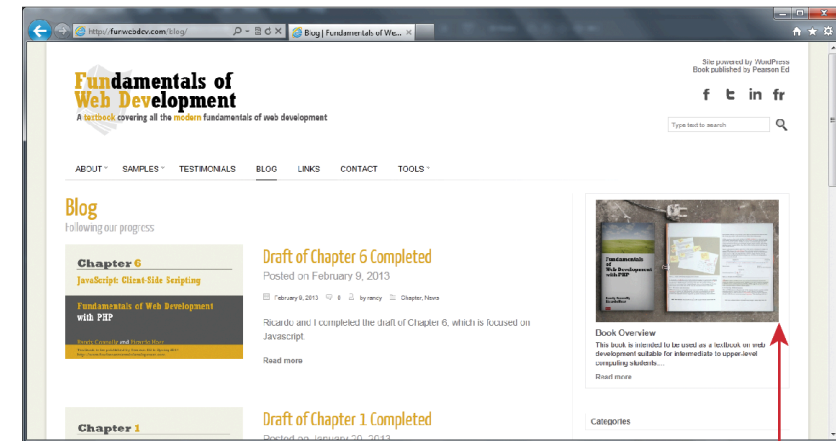
- **grid** and **flexbox** each have their strengths and these strengths can be combined
- **grid** layout is ideal for constructing the layout structure of your page
- **flexbox** is ideal for laying out the contents of a grid cell.



Responsive Design

In a **responsive design**, the page “responds” to changes in the browser size that go beyond simple percentage scaling of widths.

- smaller images will be served and
- navigation elements will be replaced as the browser window shrinks



Notice how some elements are scaled to shrink as browser window reduces in size.



When browser shrinks below a certain threshold, then layout and navigation elements change as well.

In this case, the `<ul>` list of hyperlinks changes to a `<select>` and the two-column design changes to one column.

Mobile First Design

NOTE

Mobile-first design suggests that the first step in the design and implementation of a new website should be the design and development of its mobile version (rather than as an afterthought as is often the case).

The rationale for the mobile-first approach lies not only in the increasingly larger audience whose principal technology for accessing websites is a smaller device such as a phone or a tablet.

Focusing first on the mobile platform also forces the designers and site architects to focus on the most important component of any site: the content.



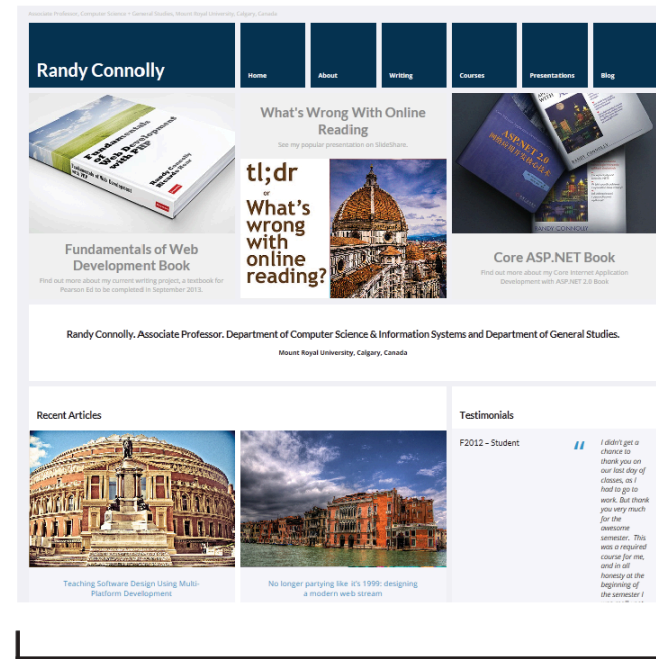
Viewports

The browser's **viewport** is the part of the browser window that displays web content.

Mobile browsers will by default scale a web page down to fit the width of the screen.

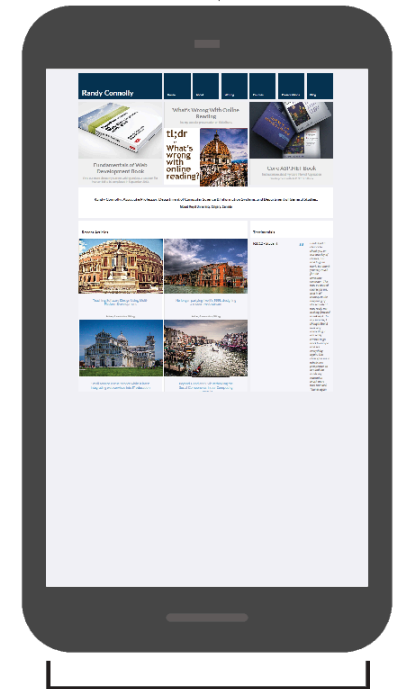
Generally, results in a viewing experience that works but is very difficult to read and use.

1 Mobile browser renders web page on its viewport



960px
Mobile browser viewport

2 It then scales the viewport to fit within its actual physical screen



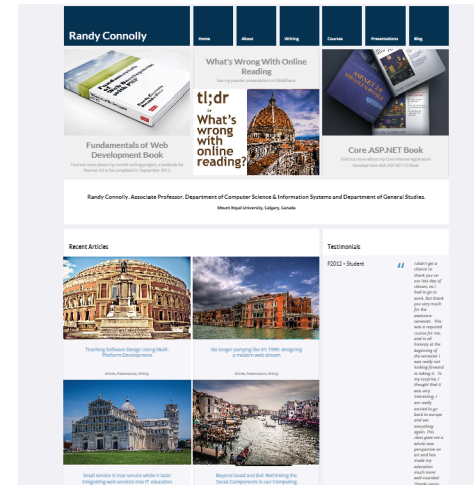
320px
Mobile browser screen

Setting Viewports

```
<html>
<head>
<meta name="viewport"
content="width=device-width, initial-scale=1" />
```

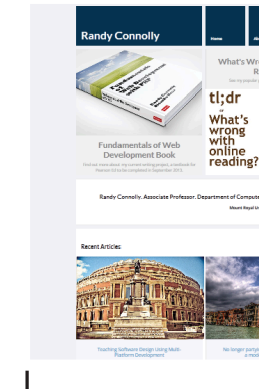
LISTING 7.3 Setting the Viewport

In Listing 7.3, the **width** attribute controls the size of the viewport, while **initialscale** sets the zoom level.



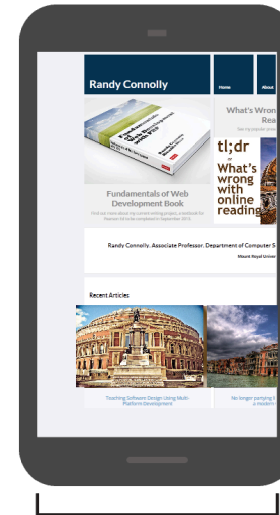
```
<meta name="viewport"
content="width=device-width, initial-scale=1" />
```

- 1 Mobile browser renders web page on its viewport and because of the <meta> setting, makes the viewport the same size as the pixel size of screen.



320px

Mobile browser viewport



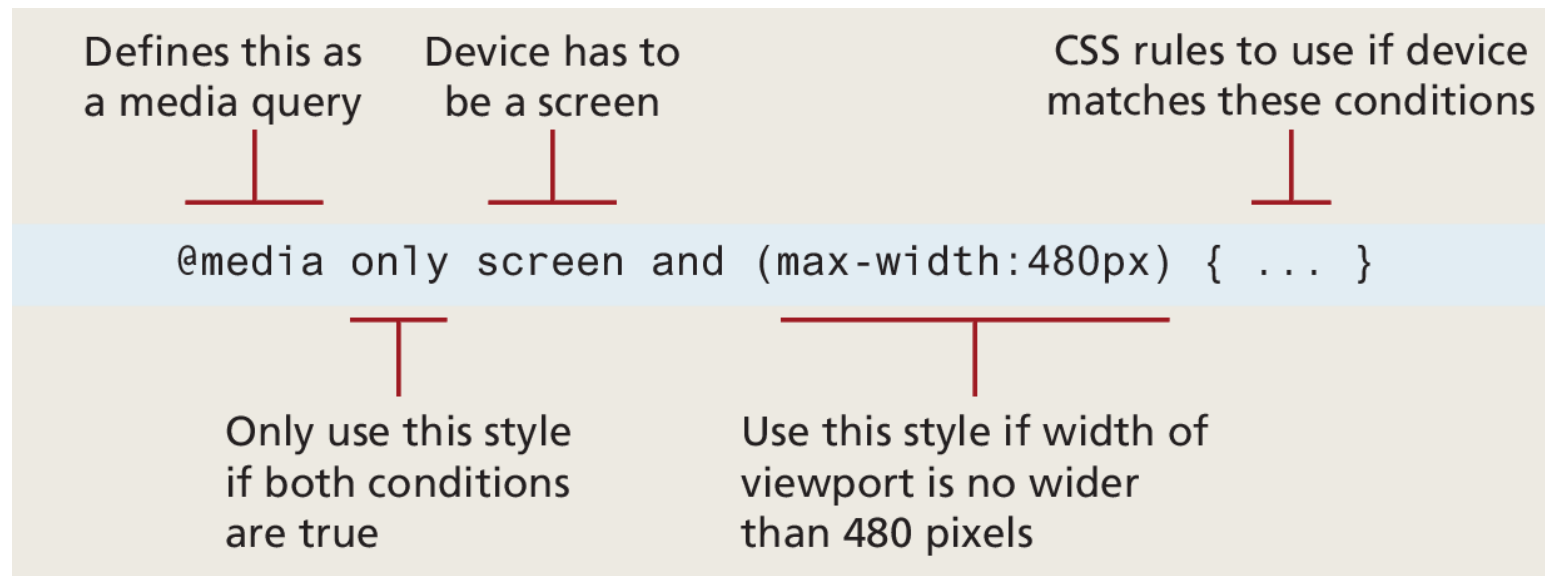
320px

- 2 It then displays it on its physical screen with no scaling.

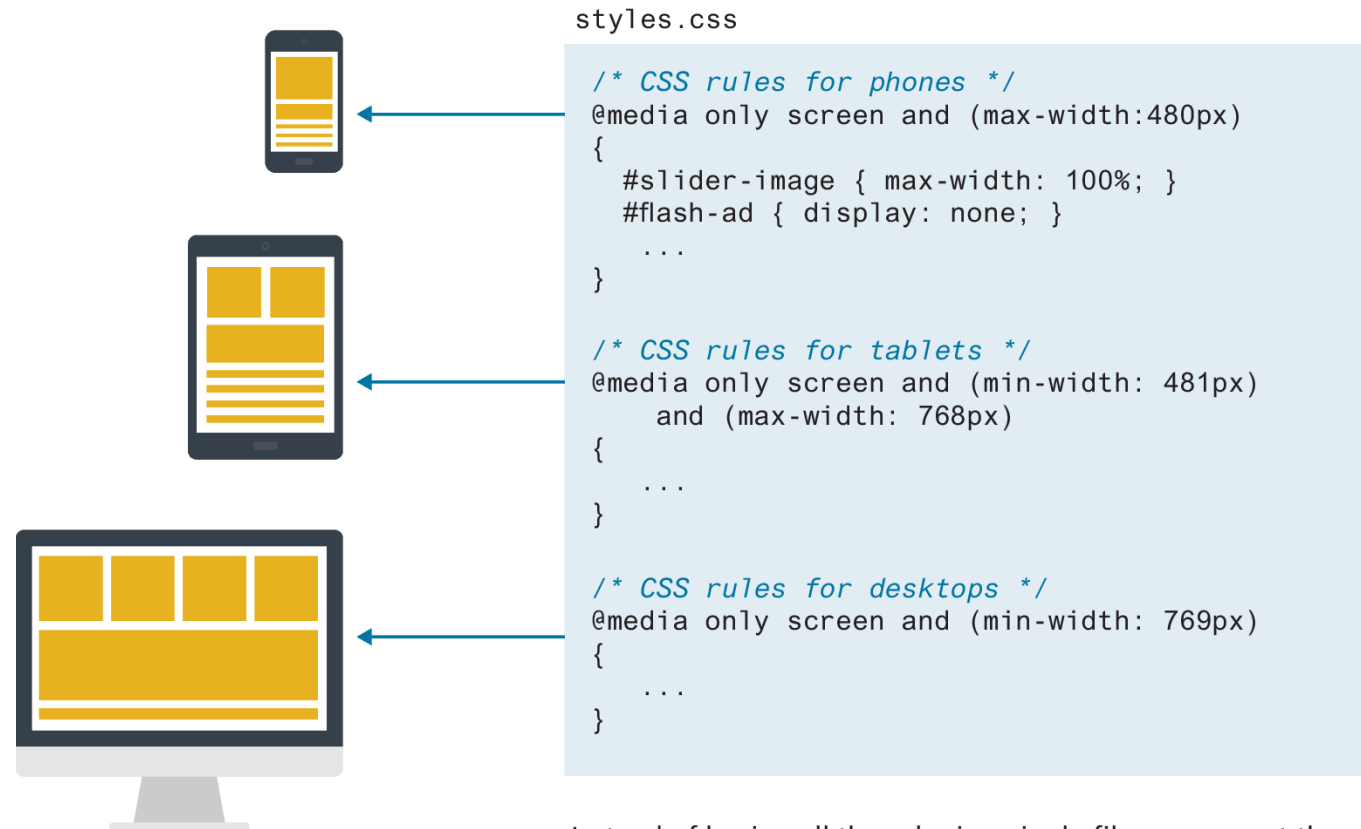
Media Queries

CSS media queries are a way to apply style rules based on the medium that is displaying the file

Contemporary responsive sites will typically provide CSS rules for phone displays first, then tablets, then desktop monitors, an approach called **progressive enhancement**



Media queries in action



Instead of having all the rules in a single file, we can put them in separate files and add media queries to <link> elements.

```

<link rel="stylesheet" href="mobile.css" media="screen and (max-width:480px)" />
<link rel="stylesheet" href="tablet.css"
      media="screen and (min-width:481px) and (max-width:768px)" />
<link rel="stylesheet" href="desktop.css" media="screen and (min-width:769px)" />
  
```


Scaling images

Making images scale in size is straightforward, but does not change the downloaded size of the image

```
img {  
    max-width: 100%;  
}
```

HTML5.1 defines the new <picture> element that lets the designer specify multiple elements. The browser determines which to use based on the viewport size.

The <picture> element

<picture>

Is this true?

```
<source media="(min-width:960px)"  
srcset="images/828-large.jpg">
```

if yes, then use this as the src for the

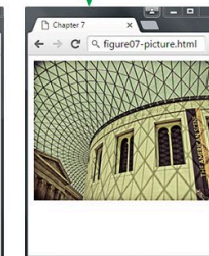
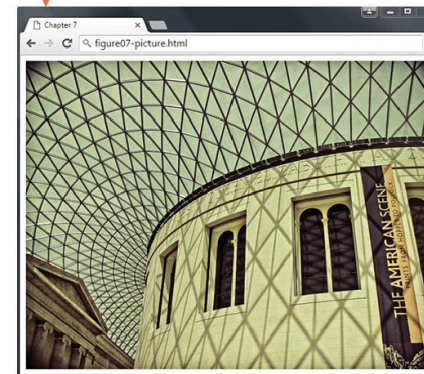
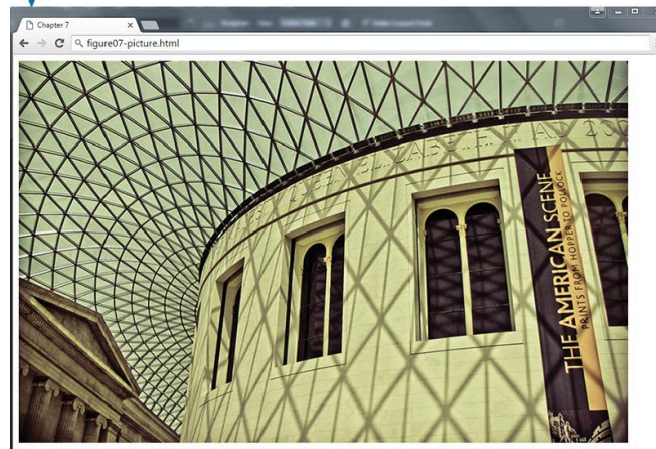
Is this true?

```
<source media="(min-width:480px)"  
srcset="images/828-medium.jpg">
```

if yes, then use this as the src for the

```
  
</picture>
```

Otherwise use the src specified in the



CSS Effects

Four CSS3 modules have become broadly popular amongst designers:

- transformations,
- filters,
- transitions, and
- animations

Transforms

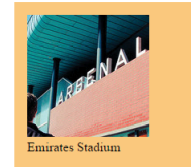
CSS **transforms** provide additional ways to change the size, position, and even the shape of HTML elements.

CSS transforms allow you to **rotate**, **skew**, **transform** (move), and **scale** an element.

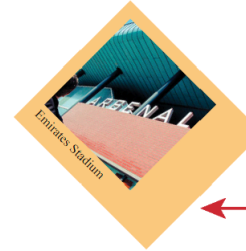
It is also possible to transform an element in 3D space using the `perspective()`, `rotate3d()`, `scale3d()`, and `translate3d()` functions

SIT Intern

```
<figure>
  
  <figcaption>Emirates Stadium</figcaption>
</figure>
```

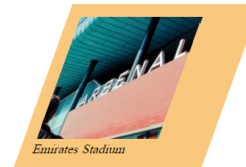


```
figure {
  padding: 1em;
  background: #FFCC80;
  width: 200px;
}
```

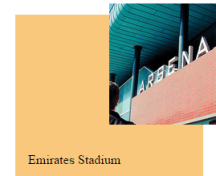


```
figure {
  transform: rotate(45deg);
}
```

Notice that the transform affects all the content within the transformed container.

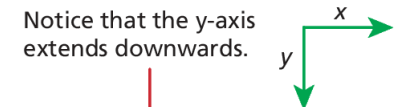


```
figure {
  transform: skew(-20deg);
}
```



```
figure img {
  transform: translateX(100px) translateY(-30px);
}
```

You can combine transforms.



```
figure {
  transform: rotate(15deg);
}
figure img {
  transform: rotate(45deg) scale(0.5);
}
```


Filters

Filters provide a way to modify how an image appears in the browser. Filters are specified by using the filter property and then one or more filter functions

```
#someImage {
  filter: grayscale(100%);
}
#anotherImage {
  /* multiple filters are space separated */
  filter: blur(5px) hue-rotate(60deg) saturate(2);
}
```

LISTING 7.4 Using a Filter



original



saturate(3)



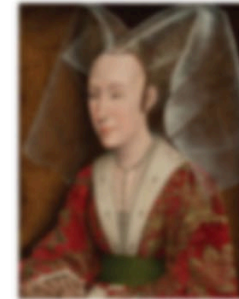
grayscale(100%)



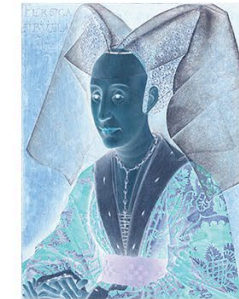
contrast(200%)



brightness(30%)



blur(3px)



invert(100%)



sepia(100%)



hue-rotate(90deg)



opacity(50%)


 brightness(1.5)
 contrast(3)
 grayscale(0.6)
 invert(0.23)
 sepia(0.2)

 brightness(1.3)
 contrast(1.1)
 hue-rotate(180deg)
 saturate(2)

Transitions

Transitions are a powerful new feature of CSS3. **Transitions** will take effect across a length of time. provide a way to indicate that a property change

- **transition-delay** The delay time in seconds before the animation begins.
- **transition-duration** How long in seconds for the transition to complete.
- **transition-property** The name of the CSS property to which the transition is applied.
- **Transition-timing-function** The function that defines how the intermediate steps in the transition are calculated. CSS defines a variety of different easing functions which define the rate of the transition.

Transition with example

1. The CSS property which will be transitioned.
2. The duration of the transition.
3. The easing function to use, which changes the speed and style of the transition(optional).
4. How long to delay before starting the transition (optional).

The button as it normally appears.

The button as it appears during transition between two states.

The button as it appears when hovered over.



```
button {
```

```
background-color: #146d37;
```

This dark green color will be displayed when the mouse is not over the button.

Which CSS property of the button is going to be transitioned across time?

1 transition-property: background-color;

We will transition the background color of the button across time.

2 How long is the transition?

```
transition-duration: 0.5s;
```

The transition will last half a second.

3 What will be the rate of transition change?

```
transition-timing-function: ease-out;
```

The transition will slow down towards the end.

4 Do we delay the start of the transition?

```
transition-delay: 0s;
```

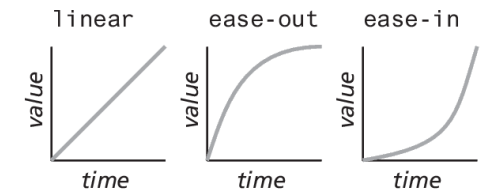
No delay (transition will start immediately)

```
}
```

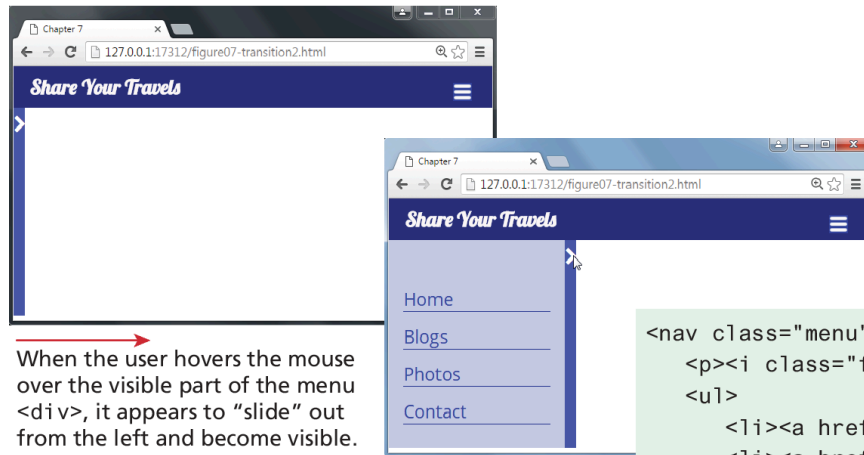
```
button:hover {
```

```
background-color: #60b946;
```

This light green color will be displayed when the mouse hovers over the button.



A sliding menu transition example



When the user hovers the mouse over the visible part of the menu <div>, it appears to “slide” out from the left and become visible.

Menu is initially hidden by being positioned outside the visible area.

```
.menu {
  position: absolute;
  left: -210px;
}
```

```
<nav class="menu">
  <p><i class="fa fa-chevron-right"></i></p>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">Blogs</a></li>
    <li><a href="#">Photos</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

```
.menu:hover {
  left: 0;
}
```

When the user hovers over the menu, move the left edge of the element to left edge of the browser (i.e., it will now be visible).

```
transition: left .6s ease-out;
```

Using the transition shorthand property. Transition the left property across 0.6 seconds and use the ease-out function (i.e., slow down transition at end).

```
.menu {
  transition: left .6s ease-out;
}
```

We want the same transition when the mouse is no longer hovering over the menu. This creates illusion of menu sliding back out of sight.

Animations

An **animation** alters one or more properties between a start state and an end state (like a transition).

Unlike transitions, animations allows designers to control the intermediate steps through **keyframe** states.



begin state

A **transition** alters one or more CSS properties across time.

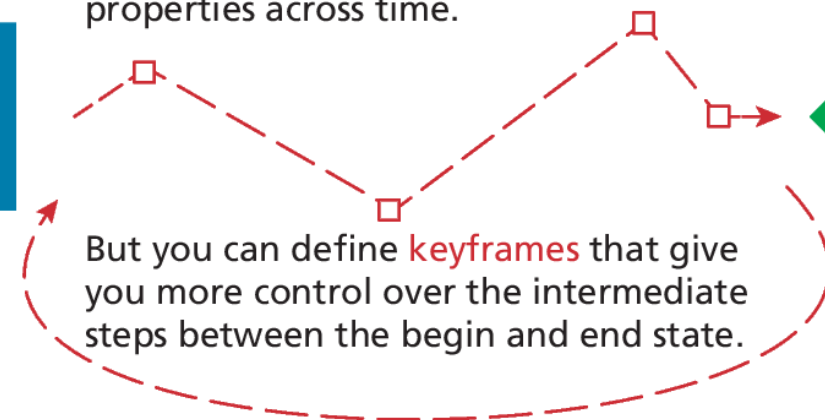
It has a begin state and then it transitions to the end state. It also needs an explicit trigger (such as hovering).



end state



An **animation** also alters one or more CSS properties across time.



But you can define **keyframes** that give you more control over the intermediate steps between the begin and end state.

No trigger is needed: an animation begins once it is defined. As well, you can loop an animation.

Animation example

```
<p class="animated">Animate Me</p>
```

```
.animated {  
  animation-iteration-count: infinite;  
  animation-name: bounceIn;  
  animation-play-state: running;  
  animation-duration: 2s;  
  animation-timing-function: ease-out;  
  animation-delay: 1s;  
}
```

| Run animation indefinitely.

| Play animation named bounceIn.

| Play animation once it is defined.

| Animation lasts 2 seconds.

| Slow animation towards the end.

| Wait a second before starting animation.

```
.animated:hover {  
  animation-play-state: paused;  
}
```

| Pause the animation by hovering over it.
(useful for debugging).

CSS preprocessors

CSS preprocessors allow the developer to write CSS that takes advantage of programming ideas such as variables, inheritance, calculations, and functions.

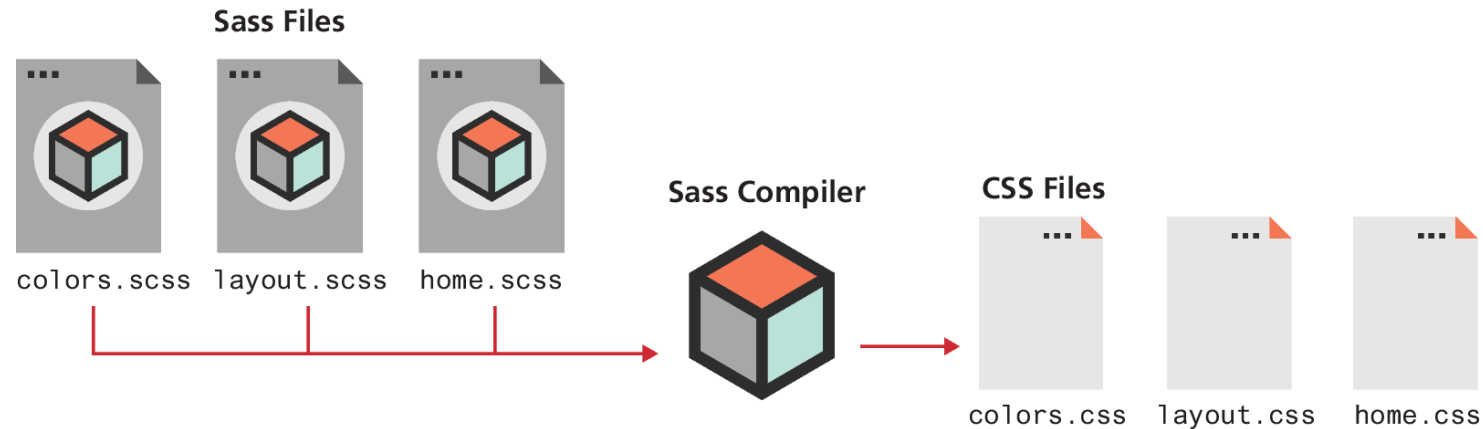
CSS itself has many limitations:

- No variables (prior to 2018).
- No encapsulation.
- No scoping rules
- No modularity. Duplication. Duplication. Duplication.

CSS preprocessor addresses these limitations. A developer can use variables, nesting, functions, or inheritance to handle duplication and avoid copy and pasting, and search and replacing.

Sass

Via command line or GUI tool, a Sass compiler compiles one or more Sass files into a regular CSS file that can be referenced in the usual way via the `<link>` element.



Sass Variables and Types

In Sass, a variable declaration looks like a property declaration in CSS.

The following code defines two Sass variables and then references them.

```
$primary-color: #647ACB;  
$spacing: 20px;  
.box {  
  background-color: $primary-color;  
  margin-top: $spacing;  
}
```

Sass nesting

- Sass provides a way to nest your styling. Listing 7.6 demonstrates how you can potentially simplify your styling using Sass nesting and variables whose scope is limited to a single block (and to its children).

```
.container {  
  .sidebar {  
    $side-color : red;  
    color: $side-color;  
    h2 {  
      color: $side-color; /* .sidebar h2 color is red */  
      ...  
    }  
  }  
}  
main {  
  ...  
  $main-color: black;  
  header {  
    h2 {  
      color: $main-color;  
      /* .sidebar main h2 color is black */  
    }  
    ...  
  }  
  article {  
    h2 {  
      color: $main-color;  
    }  
  }  
  ...  
}
```

LISTING 7.6 Using Sass nesting

Mixins and Functions

One of the key limitations of CSS is that even though there is often a lot of repetitive styling, outside the recent CSS variables there is no language feature for eliminating it.

Mixins in Sass are like a function that returns a style.

```
$mid-neutral : #9AA5B1;
$shadow : 0 0 3px 3px $mid-neutral;

@mixin card() {
  border: solid 1px $mid-neutral;
  margin: 3px;
  box-shadow: $shadow;
}

.box {
  @include card();
  background-color: white;
}

.feature {
  @include card();
}
```

This SCSS will compile into the following CSS.

```
.box {
  border: solid 1px #9AA5B1;
  margin: 3px;
  box-shadow: 0 0 3px 3px #9AA5B1;
  background-color: white;
}

.feature {
  border: solid 1px #9AA5B1;
  margin: 3px;
  box-shadow: 0 0 3px 3px #9AA5B1;
}
```

Key Terms

- absolute positioning
- animation
- BEM
- block-level elements
- box-level elements
- card
- containing block
- CSS frameworks
- CSS media queries
- CSS preprocessors
- grid layout
- filters
- fixed positioning
- flexbox layout
- flex container
- flex items
- float property
- iframe
- implicit grid
- inline elements
- keyframe
- minification
- mobile-first design
- normal flow
- positioning context
- progressive enhancement
- relative positioning
- responsive design
- Sass
- style guide
- transforms
- transitions
- UI design tool
- viewport

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.